

AI-Based Food Recognition with Nutrition-Aware Recommendations

A Report submitted
in partial fulfillment of the requirement
for the award of the Degree of

Bachelor of Technology

in

Computer Science and Engineering

by

Jaspreet Singh Enrollment No.: 2022BCSE019

Inderpal Singh Enrollment No.: 2022BCSE037

Under the guidance of

Dr. Lavanya Madhuri Bollipo

Supervisor

Dr. Insha Majeed

Co-Supervisor

Department of Computer Science and Engineering
National Institute of Technology Srinagar
Kashmir 190006, INDIA

June 2026

CERTIFICATE

This is to certify that the project report entitled **AI-Based Food Recognition with Nutrition-Aware Recommendations** submitted by **Jaspreet Singh (2022BCSE019)** and **Inderpal Singh (2022BCSE037)** to the Department of Computer Science and Engineering, National Institute of Technology Srinagar, in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a bona fide record of project work carried out under the supervision of Dr. Lavanya Madhuri Bollipo and co-supervision of Dr. Insha Majeed.

The work presented in this report has been prepared as a final year project in the area of computer vision, full-stack web engineering, nutrition data integration, and recommendation systems. To the best of our knowledge, the work described here has not been submitted elsewhere for the award of any degree or diploma.

Dr. Lavanya Madhuri Bollipo

Supervisor

Department of Computer Science and
Engineering

National Institute of Technology Srinagar

Dr. Insha Majeed

Co-Supervisor

Department of Computer Science and
Engineering

National Institute of Technology Srinagar

STUDENT DECLARATION

We declare that this project report titled **AI-Based Food Recognition with Nutrition-Aware Recommendations** is submitted in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering. The project is a record of original work carried out by us under the supervision of Dr. Lavanya Madhuri Bollipo and co-supervision of Dr. Insha Majeed.

The matter embodied in this project, in full or in part, has not been submitted to any other institution or university for the award of any degree or diploma. We also declare that the work submitted by us is original and that all external datasets, software libraries, documentation sources, and research references used in the project have been acknowledged appropriately.

Jaspreet Singh

2022BCSE019

Inderpal Singh

2022BCSE037

Srinagar, 190006

June 2026

ACKNOWLEDGEMENTS

We express our sincere gratitude to our project guide, Dr. Lavanya Madhuri Bollipo, and co-supervisor, Dr. Insha Majeed, for their guidance, review, and technical direction throughout this final year project. Their feedback was especially useful when the work moved from a broad idea into practical problems such as cleaning dataset labels, checking food-class images, correcting nutrition mappings, and connecting the model output to a usable web interface.

We are thankful to the Department of Computer Science and Engineering, National Institute of Technology Srinagar, for providing the academic environment and opportunity to carry out this work. We also acknowledge the open-source communities behind Python, FastAPI, Next.js, Ultralytics YOLO, PyTorch, and the scientific computing tools used in this project. The availability of these tools allowed us to spend more time on project-specific issues such as class normalization, nutrition validation, and interface behavior.

We also thank our families, friends, and peers for their support, feedback, and patience during the design, implementation, debugging, testing, and documentation phases of the work.

Jaspreet Singh
2022BCSE019

Inderpal Singh
2022BCSE037

ABSTRACT

Manual food logging often fails for a simple reason: the same work has to be repeated after every meal. A user has to identify each dish, estimate the quantity, search for a suitable database entry, and then repeat the process for the next item on the plate. This becomes more difficult for Indian meals because one plate can contain rice, dal, chutney, curry, fried snacks, sweets, and beverages, sometimes with regional or household-specific names. This project addresses that practical problem by developing an AI-based food recognition system with nutrition-aware recommendations. The system accepts a food image, detects visible food items using a YOLO-based object detector, maps the detected labels to Indian nutrition records, estimates calories and macronutrients per 100 grams, and presents goal-aware dietary suggestions through a web application.

The implemented prototype combines a FastAPI backend, a Next.js frontend, a SQLite nutrition database built mainly from the Indian Nutrient Database, and a merged YOLO-format dataset containing 72 canonical Indian food classes. Five source datasets were consolidated through a normalization pipeline that resolves spelling variants, regional labels, and duplicate class names. This step required careful checking because small naming differences such as idly and idli, or satham and rice, can become separate detector classes if they are not handled before training. The final balanced dataset contains 48,693 exported image-label pairs across training, validation, and test splits. Visual verification sheets were generated from real dataset images, confirming that all 72 class names have corresponding samples while also revealing a small number of annotation issues that should be corrected in later training cycles.

The final detector training was completed with YOLO11s on the 72-class dataset. The best model achieved 0.830 mAP@50 and 0.692 mAP@50:95 on validation, and 0.820 mAP@50 and 0.680 mAP@50:95 on the held-out test split. The nutrition layer maps all 72 dataset classes to database entries by using explicit semantic mappings and verified supplemental rows before fuzzy matching. This design decision was taken after observing that simple fuzzy matching can return a food with a similar name but a different nutritional profile. The backend exposes endpoints for image analysis, nutri-

tion validation, macro calculation, daily goals, health conditions, and recommendation generation. The frontend supports image upload, detection visualization, serving adjustment, macro tracking, and personalized guidance. The report presents the design decisions, implementation process, validation evidence, limitations, and possible future improvements of the project.

Contents

CERTIFICATE	i
STUDENT DECLARATION	iii
ACKNOWLEDGEMENTS	v
ABSTRACT	vii
LIST OF FIGURES	xiii
LIST OF TABLES	xv
ABBREVIATIONS AND NOTATIONS	xvii
1 INTRODUCTION	1
1.1 Overview	1
1.2 Motivation	2
1.3 Problem Statement	2
1.4 Objectives	3
1.5 Scope of the Project	3
1.6 Major Contributions	4
1.7 Report Organization	5
2 LITERATURE REVIEW	7
2.1 Scope of the Review	7
2.2 Paper-wise Review	7
2.2.1 IndianFoodNet: Detecting Indian Food Items Using Deep Learning	7
2.2.2 Khana: A Comprehensive Indian Cuisine Dataset	8
2.2.3 What is there in an Indian Thali?	8
2.2.4 Single-item Training for Multi-dish Recognition	9

2.2.5	Lightweight Real-time Food Calorie Estimation Using CNNs	9
2.2.6	Indian Food Image Classification with MobileNetV3	9
2.2.7	Development of an Indian Food Composition Database	10
2.2.8	Synergistic Frameworks for Indian Food Computing	10
2.2.9	Enhancing FKG.in	10
2.2.10	Label AI	11
2.2.11	Nutrition Information Estimation from Food Photos	11
2.2.12	NutriGen	11
2.2.13	LLMs and Computer Vision for Personalized Nutrition Advice	12
2.2.14	AI-driven Personalized Meal Planning	12
2.2.15	Diet Engine	12
2.2.16	Smart Diet Planner	13
2.2.17	Evaluation of ChatGPT for Nutrient Estimation from Meal Photographs	13
2.2.18	Indian Food Classification and Dietary Recommendations Using CNNs	13
2.3	Comparative Summary	14
2.4	Research Gap	17
3	METHODOLOGY	19
3.1	Methodological Overview	19
3.2	YOLO Detection Approach	20
3.3	Source Dataset Collection	21
3.4	Canonical Class Normalization	21
3.5	YOLO Label Representation	22
3.6	Dataset Splitting and Augmentation	22
3.7	Nutrition Database Methodology	23
3.8	Nutrition Mapping Methodology	24
3.9	Macro Calculation Methodology	24
3.10	Validation Methodology	25
4	SYSTEM DESIGN	27
4.1	Design Goals	27
4.2	High-Level Architecture	27
4.3	Runtime Data Flow	28
4.4	Backend Design	29

4.5	Frontend Design	30
4.6	Database Design	30
4.7	API Design	31
4.8	Security and Validation Design	32
4.9	Design Trade-Offs	33
5	IMPLEMENTATION	35
5.1	Development Environment	35
5.2	Dataset Build Script	35
5.3	Final YOLO Training Notebook	36
5.4	Visual Class Verification Implementation	36
5.5	Nutrition Database Build Implementation	37
5.6	Vision Service Implementation	37
5.7	Nutrition Service Implementation	38
5.8	Recommendation and Macro Implementation	38
5.9	Frontend Implementation	38
5.10	Code Organization	39
5.11	Implementation Challenges	40
6	EXPERIMENTS AND VALIDATION	41
6.1	Experimental Scope	41
6.2	Dataset Validation	42
6.3	YOLO Training and Detector Evaluation	42
6.4	Class Verification Experiment	46
6.5	Nutrition Database Validation	46
6.6	Nutrition Mapping Coverage Experiment	47
6.7	Service Lookup Validation	48
6.8	Backend API Validation	48
6.9	Frontend Workflow Validation	49
6.10	Macro Calculation Validation	49
6.11	Experimental Findings	50
7	RESULTS	51
7.1	Dataset Engineering Results	51
7.2	Detector Training Results	52
7.3	Nutrition Mapping Results	52
7.4	Application Results	53

7.5	Macro and Recommendation Results	53
7.6	Result Synthesis	54
8	DISCUSSION	55
8.1	Interpretation of Results	55
8.2	Strengths of the System	55
8.3	Limitations	56
8.4	Ethical and Practical Considerations	57
8.5	Why Supplemental Rows Were Added	57
8.6	Lessons Learned	58
9	CONCLUSION	59
10	FUTURE WORK	61
10.1	Self-Trained Recommendation Model	61
10.2	Portion Size Estimation	61
10.3	Mobile Deployment	62
10.4	Personalized Long-Term Tracking	62
10.5	Clinical Safety and Explainability	62
A	DATASET CLASSES AND NUTRITION MAPPINGS	67
A.1	Expanded Dataset Class List	67
A.2	Class-to-INDB Mapping Table	67
A.3	Known Annotation Issues	70
A.4	Visual Verification Sheet Set	71
A.5	Per-Class Dataset Counts	86
B	API RESPONSE EXAMPLES	91
B.1	Analyze Food Response	91
B.2	Macro Calculation Request	92
B.3	Macro Calculation Response	92
B.4	No Food Response	93
B.5	Macro Target Reference Tables	93

List of Figures

1.1	End-to-end workflow of the implemented food recognition and nutrition system	2
3.1	Methodological workflow for dataset, model, nutrition, backend, and frontend integration	20
4.1	Layered architecture of the food recognition and nutrition system	28
4.2	Runtime request-response flow	29
4.3	Entity relationship used for class-to-nutrition lookup	31
5.1	Frontend detection overlay showing Bhatura and Chole detections . . .	39
6.1	Verified system-level coverage used for validation	41
6.2	YOLO11s final training curves from the resumed 72-class run	43
6.3	Ultralytics result plot for the final YOLO11s run	44
6.4	Normalized confusion matrix for the YOLO11s held-out test evaluation	45
6.5	Sample YOLO11s predictions from the held-out test split	45
6.6	Frontend workflow checks used during qualitative validation	49
A.1	Visual verification half-sheet 1	71
A.2	Visual verification half-sheet 2	72
A.3	Visual verification half-sheet 3	73
A.4	Visual verification half-sheet 4	74
A.5	Visual verification half-sheet 5	75
A.6	Visual verification half-sheet 6	76
A.7	Visual verification half-sheet 7	77
A.8	Visual verification half-sheet 8	78
A.9	Visual verification half-sheet 9	79
A.10	Visual verification half-sheet 10	80
A.11	Visual verification half-sheet 11	81

A.12 Visual verification half-sheet 12	82
A.13 Visual verification half-sheet 13	83
A.14 Visual verification half-sheet 14	84
A.15 Visual verification half-sheet 15	85
A.16 Visual verification half-sheet 16	86

List of Tables

2.1	Portrait summary of the supplied literature	14
3.1	Source datasets used for merged Indian food dataset	21
3.2	Examples of raw-to-canonical label normalization	22
4.1	Runtime nutrition database tables	30
4.2	Implemented backend endpoints	32
5.1	Important implementation files	40
6.1	Generated dataset summary	42
6.2	Final YOLO11s training setup	43
6.3	Final YOLO11s aggregate detector metrics	44
6.4	Visual class verification result	46
6.5	Nutrition database validation result	47
6.6	Nutrition mapping coverage for 72-class dataset	47
6.7	Verified supplemental nutrition rows	48
6.8	Corrected nutrition lookup examples	48
7.1	Representative categories in the 72-class dataset	51
7.2	Examples of improved nutrition mappings	53
A.1	Full 72-class dataset to INDB mapping table	67
A.2	Sample-level annotation issues found during visual verification	70
A.3	Per-class dataset counts after merge and train augmentation	87
B.1	Weight-loss macro targets for 2,000 kcal	93
B.2	Muscle-gain macro targets for 2,000 kcal	94
B.3	Maintenance macro targets for 2,000 kcal	94
B.4	Endurance macro targets for 2,000 kcal	95

ABBREVIATIONS AND NOTATIONS

Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BMR	Basal Metabolic Rate
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
FYP	Final Year Project
HTTP	Hypertext Transfer Protocol
INDB	Indian Nutrient Database
IoU	Intersection over Union
JSON	JavaScript Object Notation
LLM	Large Language Model
ML	Machine Learning
NMS	Non-Maximum Suppression
ORM	Object Relational Mapping
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
YOLO	You Only Look Once

Notations

B	Bounding box represented as center coordinates, width, and height.
C	Set of canonical food classes in the merged dataset.
P	Predicted probability or confidence score produced by the detector.
IoU	Area overlap ratio between predicted and ground-truth bounding boxes.
M_c	Nutrition mapping selected for class c .
E	Energy value in kilocalories per 100 grams.

Chapter 1

INTRODUCTION

1.1 Overview

Food logging appears simple at first, but it becomes inconvenient when it has to be done every day. A person has to remember what was eaten, estimate the quantity, search for a matching food entry, and repeat the same process for each item in the meal. In Indian meals this task is even less direct. A single plate may contain rice, dal, chutney, curry, a fried snack, and a sweet in small portions. The user may remember the meal clearly, but most logging applications still expect every item to be searched and entered separately.

Computer vision can reduce this effort, although it does not solve the complete nutrition problem on its own. Food does not have a fixed shape, lighting changes its appearance, and many items are partly hidden by bowls, plates, or other food items. Indian food adds another difficulty because many dishes share ingredients and colours. For example, `aloo_gobi`, `dum_aloo`, and `aloo_masala` can all contain potato-heavy textures, while chutneys may appear only as small side portions. During development it became clear that a useful system cannot stop after producing a confident visual label. That label must also be connected to a nutrition record that is reasonable for the dish.

This project implements an AI-based food recognition system with nutrition-aware recommendations. The implemented system combines a YOLO object detector, a curated Indian food dataset, a SQLite nutrition database derived mainly from the Indian Nutrient Database, a FastAPI backend, and a Next.js frontend. It accepts a food image, detects visible food objects, maps each class to a nutrition record, calculates macronutrients, and generates dietary suggestions using the user's daily goal and selected health context.

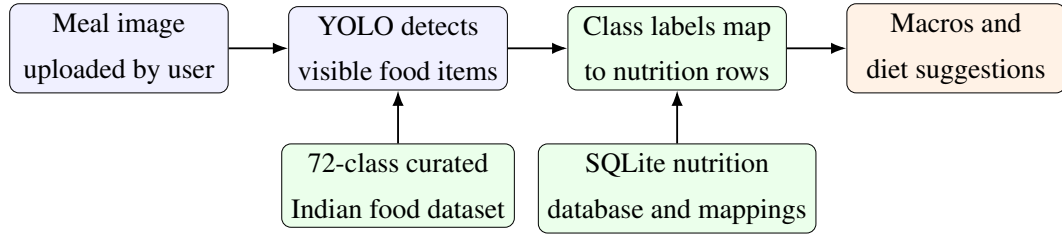


Figure 1.1: End-to-end workflow of the implemented food recognition and nutrition system

1.2 Motivation

The motivation for this work came from three practical needs observed while studying existing food logging workflows. First, food logging has to be fast enough to fit into daily life. If a user must search and type after every meal, the habit usually breaks after a short period. Second, Indian food recognition needs domain-specific labels and mapping logic. A general detector may identify a bowl or plate, but it will not reliably distinguish *rajma_curry*, *sambar*, *poha*, *thepla*, or *bhakarwadi*. Third, nutrition advice only makes sense when user context is considered. A calorie-dense meal may be acceptable for a muscle-gain goal but unsuitable for weight loss; similarly, a carbohydrate-heavy meal needs different guidance for a diabetic user than for an endurance-focused user.

A second motivation emerged during implementation. Detection and nutrition were found to be separate problems. A model may return *samosa* or *palak_paneer*, but the system still has to connect that label to the correct database row. If the mapping is wrong, the interface may display values that look precise but are nutritionally incorrect. During testing, weak fuzzy matching produced cases such as *palak_paneer* being mapped to a generic cottage-cheese entry instead of spinach paneer. This issue changed the direction of the project. Nutrition mapping was treated as a core engineering task rather than a small post-processing step.

1.3 Problem Statement

The problem addressed in this project is to design and implement a full-stack system that can:

1. Detect one or more Indian food items from an uploaded image.
2. Normalize detected class labels into stable canonical food names.
3. Map detected foods to nutrition database records with maximum safe coverage.

4. Estimate calories, protein, carbohydrates, and fat per detected item.
5. Provide personalized dietary recommendations using user goals and health conditions.
6. Present the results through an accessible web interface with bounding boxes, serving adjustment, and macro tracking.

The problem is therefore not limited to model training. It includes dataset engineering, data validation, backend API design, frontend interaction design, and nutrition-aware business logic. Each part affects the reliability of the final output. For example, a correct frontend cannot compensate for a wrong food-to-nutrition mapping, and a good detector still becomes less useful if the class labels are inconsistent.

1.4 Objectives

The main objectives of the project are:

- To build a canonical Indian food dataset by merging multiple YOLO-format source datasets.
- To resolve duplicate and inconsistent food labels into a stable set of classes.
- To integrate a YOLO-based detection service into a FastAPI backend.
- To create a nutrition lookup service backed by a local SQLite database.
- To improve nutrition mapping quality through explicit class-to-database mappings.
- To implement macro calculation for different dietary goals and health profiles.
- To develop a Next.js frontend that supports image upload, detection visualization, nutrition cards, and recommendation display.
- To validate the dataset, class names, nutrition mappings, and service behavior.

1.5 Scope of the Project

The scope of the implemented prototype is limited to image-based recognition of Indian food classes present in the curated dataset. The model output is treated as a detected dish label, and nutrition values are reported per 100 grams unless the user adjusts serving

assumptions in the frontend. The system does not perform physical volume estimation or exact portion-size measurement from a single image. This boundary was kept deliberately because reliable portion estimation would need depth, scale, segmentation, or controlled capture conditions. The prototype therefore focuses on a practical nutrition-aware workflow that can later be extended with portion estimation, mobile deployment, and larger nutrition databases.

The backend and frontend are designed for local full-stack deployment. The backend exposes REST-style endpoints for analysis, macro calculation, validation, and recommendation generation. The frontend is built as a single-page user experience for uploading images and reviewing results. The recommendation module uses Groq-backed language model generation when a key is configured, with structured inputs from the detected foods and user profile. This makes the recommendation layer usable in the prototype while keeping the nutrition arithmetic inside the backend.

1.6 Major Contributions

The major contributions of this project are summarized below. They are written from the implemented system rather than only from the model-training stage, because the final output depends on the complete pipeline.

1. A merged Indian food dataset containing 72 canonical classes and 48,693 exported image-label pairs.
2. A normalization pipeline that maps raw labels such as AlooGobi, aloo gobhi, thengai chutney, satham, and medu vadai into stable canonical labels.
3. Visual class verification artifacts showing real image samples for all 72 classes.
4. A nutrition database build process that imports INDB nutrition fields and creates a runtime SQLite database.
5. A safer nutrition mapping strategy that covers all 72 dataset classes using explicit semantic mappings and verified supplemental nutrition rows before fuzzy matching.
6. A FastAPI analysis endpoint that combines detection results, bounding boxes, nutrition data, and food-not-found handling.
7. A frontend interface that combines upload, detection visualization, serving adjustment, macro progress, and recommendations.

1.7 Report Organization

The report is organized in the same order in which the project was developed. Chapter 2 reviews the supplied research papers and identifies the gap addressed by the implementation. Chapter 3 describes dataset construction, nutrition mapping, macro logic, and validation strategy. Chapter 4 presents the system architecture. Chapter 5 explains implementation details and development challenges. Chapter 6 covers experiments and validation. Chapter 7 presents the results. Chapter 8 discusses limitations and interpretation of results. Chapters 9 and 10 provide the conclusion and future work. The appendices include detailed class mappings, API examples, and supporting verification material.

Chapter 2

LITERATURE REVIEW

2.1 Scope of the Review

This review is restricted to the research papers supplied with the project material. The papers cover Indian food image datasets, YOLO and CNN-based recognition, segmentation for thali-style meals, Indian nutrition databases, food-name mapping, barcode-based nutrition systems, LLM-assisted meal planning, and the reliability limits of direct vision-language nutrient estimation. Papers centred on 3D volume or depth estimation are intentionally excluded because the implemented system reports per-100g macro values and does not attempt monocular volume measurement.

The review is written around the actual problem faced in the project. A food image has to be converted into detected classes, those classes have to be matched to reliable nutrition rows, and the result has to be useful enough for a recommendation workflow. This helped keep the reading focused. Instead of treating computer vision, databases, and recommendations as separate topics, the papers were compared according to how much they helped in designing the implemented pipeline.

2.2 Paper-wise Review

2.2.1 IndianFoodNet: Detecting Indian Food Items Using Deep Learning

IndianFoodNet focuses on detecting Indian food items with YOLO-style object detection rather than classifying a full image as a single dish [1]. This was relevant because Indian meals often contain multiple items in the same photograph. A detector can mark each visible item with a bounding box and a class label, which is more useful than a single

top-level prediction for a whole plate.

The paper provided a clear reason to use a custom food detector for Indian cuisine. However, it also showed a boundary that became important during implementation. Detection alone does not complete nutrition tracking. A correct label such as `samosa` or `idli` still has to be connected to a database row containing calories, protein, carbohydrates, and fat. The implemented project therefore extends the detection stage with a separate mapping and nutrition layer.

2.2.2 Khana: A Comprehensive Indian Cuisine Dataset

The Khana dataset study addresses the lack of large Indian food image benchmarks by presenting a larger collection of Indian cuisine images across 80 classes [2]. The work compares classification backbones such as ResNet, Vision Transformer, and ConvNeXT, and reports ConvNeXT-S as the best-performing model in the supplied summary.

The main value of this paper for the project was its discussion of data quality and class coverage. Indian foods have high visual similarity, strong regional variation, and class imbalance. Similar issues appeared in the merged dataset prepared for this work, where visually close dishes and rare classes required careful normalization and train-only augmentation. The limitation is that Khana is mainly a single-label classification benchmark. The implemented system needed multi-item detection, so the data-related lesson was adopted while the deployed task remained YOLO object detection.

2.2.3 What is there in an Indian Thali?

The Indian Thali paper studies the cluttered and overlapping nature of thali meals [3]. It uses segmentation and prototype matching to identify food regions in a controlled multi-camera setup. This paper was useful because it describes a difficulty that also appears in ordinary meal photographs: curries, rice, dal, chutneys, and fried items may touch or overlap, making boundaries unclear.

The implemented system uses a more lightweight input setting. It accepts a single uploaded image and predicts YOLO bounding boxes instead of relying on a kiosk or multi-view camera hardware. This does not provide the same boundary detail as segmentation, but it is easier for a normal user to operate. The paper therefore serves mainly as a reference for future improvement, especially if the project later adds instance segmentation for mixed plates.

2.2.4 Single-item Training for Multi-dish Recognition

The single-item training paper studies multi-dish Indian food recognition using a class-agnostic framework based on segmentation and DenseNet-style feature extraction [4]. Its main contribution is the attempt to generalize from individual dish examples to more complex platter scenes. This is a realistic concern because gathering well-annotated multi-dish Indian meal data takes more time than collecting single-food images.

The paper indicates that stronger feature learning can help in multi-food scenes. At the same time, its two-stage design is less convenient for a responsive web application. For the current implementation, a single-stage YOLO pipeline was selected so that localization and classification happen in one inference pass. This keeps the backend easier to integrate and makes API usage more practical on ordinary development hardware.

2.2.5 Lightweight Real-time Food Calorie Estimation Using CNNs

The lightweight CNN paper argues for low-latency food recognition models in nutrition applications [5]. The supplied summary reports a parameter-optimized CNN with useful validation accuracy and fast inference on common food datasets. The broader lesson is practical: a dietary tool should not feel slow, because users are unlikely to wait through a heavy model every time they log a meal.

This influenced the preference for a compact detector and a web API design where image inference, nutrition lookup, and response generation are kept as separate services. The paper is less aligned with the Indian-food requirement because its datasets are mostly generic or Western. That gap is handled through Indian food datasets and a 72-class canonical label set.

2.2.6 Indian Food Image Classification with MobileNetV3

The MobileNetV3 transfer learning paper applies data augmentation and pretrained mobile-friendly models to Indian food classification [6]. It is relevant because it shows that smaller models can still perform well when training data is limited, provided augmentation is handled carefully.

The same general idea is applied here at the object detection level. Augmentation is applied only to the training split so that synthetic variants do not leak into validation or test evaluation. The limitation of the MobileNetV3 study is its narrow class range, which is not enough for a realistic Indian meal assistant. The implemented dataset therefore expands the class coverage and uses detection so that multiple foods can be handled in

one image.

2.2.7 Development of an Indian Food Composition Database

The INDB paper is central to the nutrition layer of this project [7]. It focuses on cooked Indian recipes and highlights why raw ingredient tables are often inadequate for estimating food macros. Cooking method, oil absorption, water loss, and recipe composition can change the final nutrient profile substantially.

For this project, the INDB spreadsheet is converted into a SQLite database used by the FastAPI backend. This gives the system a deterministic source for calories and macronutrients instead of asking a language model to estimate numerical values. The remaining challenge is coverage. Some YOLO classes were absent from the available database sheet, so verified supplemental rows were added with source metadata rather than forcing unsafe fuzzy matches.

2.2.8 Synergistic Frameworks for Indian Food Computing

The synergistic frameworks paper describes the compatibility problem between visual food datasets and nutritional databases [8]. A model label and a database food name may refer to the same dish but still differ in spelling, language, word order, or regional naming. This is a common source of failure in food computing systems because the model may be visually correct while the nutrition lookup fails.

This idea was directly observed during backend development. The project uses normalized class names, manual semantic mappings, source-backed supplemental rows, and conservative fuzzy matching. This treats mapping as a major part of the system rather than a small string-matching detail. The paper also supports the decision to store mappings in the database build output so that they can be inspected later.

2.2.9 Enhancing FKG.in

The FKG.in paper uses large language models and food knowledge graphs to parse regional recipe text and derive structured nutritional information [9]. Its strength is that it can work with unstructured recipe descriptions, which are common in Indian cooking because recipes are often written informally and vary by household.

This implementation uses a different entry point. Instead of asking the user to type or paste a recipe, the system begins with a food photograph. YOLO provides the initial food labels, and the nutrition database provides macro values. The LLM layer

is used later for recommendation text, not for creating the base nutrient values. This separation was important because numerical macro values should come from a traceable data source.

2.2.10 Label AI

Label AI studies barcode scanning and nutrition mapping for packaged foods [10]. It uses product codes and external food data to support fitness planning and packaged-food assessment. This kind of system works well when the food has a barcode and a standard label.

Indian home-cooked meals usually do not fit that scenario. A serving of rajma, poha, idli, dosa, or a thali has no barcode, and the user may not know the exact ingredients. The implemented system replaces barcode input with visual detection and a local nutrition database. The comparison is useful because it shows that nutrition lookup is easier when a reliable identifier already exists. In this project, the identifier has to be inferred from the image and then checked against a food database.

2.2.11 Nutrition Information Estimation from Food Photos

The multi-dataset nutrition estimation paper predicts food or ingredient attributes from photos and then aggregates nutrition using structured data [11]. This is close to the logic of the current system because both approaches avoid treating calorie estimation as a pure image-regression problem.

The difference is in scope. Ingredient-level prediction is more detailed, but it is also harder to validate, especially when serving size and food mass are unknown. This work uses class-level macro lookup per 100g and lets the user adjust servings. This is a conservative choice. It provides useful macro context while avoiding claims about exact plate weight from a single 2D photograph.

2.2.12 NutriGen

NutriGen explores personalized meal-plan generation with large language models under nutritional constraints [12]. It supports the idea that LLMs can be useful in diet applications when the prompt contains user goals, macro targets, restrictions, and other structured context.

The implementation uses this lesson but changes the data flow. Instead of relying only on a manually entered profile, the recommendation service receives detected foods

and calculated macros from the backend. This means the generated advice can respond to what the user actually uploaded. The LLM is therefore used as an explanation and recommendation layer, while the backend remains responsible for nutrition arithmetic.

2.2.13 LLMs and Computer Vision for Personalized Nutrition Advice

The LLM and computer vision paper combines OCR with language models to read printed nutrition labels and generate advice [13]. It demonstrates a useful pattern: computer vision should extract grounded information first, and the language model should reason from that extracted information.

The same pattern is followed here, but OCR is replaced with food-object detection. The backend extracts class labels and macro values, then passes structured context into the recommendation layer. This is more suitable for cooked meals because there is usually no printed label to read. The limitation of the OCR-based approach is therefore the point where food-image detection becomes useful.

2.2.14 AI-driven Personalized Meal Planning

The AI-driven meal planning paper describes a web-based platform for tailoring diet plans to user goals and health constraints [14]. Its relevance is not only the use of generative AI, but also the full-stack nature of the system. Users need an interface, profile settings, dietary targets, and suggestions they can understand.

The application developed here adopts this product-level view. The frontend asks for goals and health conditions, displays detected foods and macro totals, and then requests personalized advice. The difference is that this system visually checks the uploaded meal before recommendations are generated. This makes the advice more grounded than a plan generated from profile data alone.

2.2.15 Diet Engine

Diet Engine combines YOLOv8 food detection with an NLP chatbot for real-time dietary guidance [15]. It is one of the closest papers to the implemented architecture because it joins object detection with conversational recommendation.

The project follows the same general loop but adapts it to Indian food. The dataset, class names, and nutrition mappings are treated as domain-specific engineering problems. The supplied summary also notes weak generalization to Indian foods in Diet

Engine, which supports the decision to use Indian food datasets and verified class-to-database mappings instead of relying on generic food categories.

2.2.16 Smart Diet Planner

Smart Diet Planner presents a web-based nutrition management system using a Python backend, database-backed food records, and health calculations such as BMR or TDEE [16]. It is relevant because it treats diet planning as a software workflow, not only as a model prediction task.

The implemented system shares the backend-plus-database pattern but adds a computer vision entry point. Manual logging is still useful for controlled diet tracking, but it can become tedious. By using image upload first and serving adjustment second, the system reduces some user effort without claiming that the camera can estimate every detail automatically.

2.2.17 Evaluation of ChatGPT for Nutrient Estimation from Meal Photographs

The ChatGPT nutrient-estimation evaluation is important because it reports a practical limitation of vision-language models [17]. Such models may identify foods reasonably well, but the supplied summary notes poor agreement for exact nutrient values. This matters because fluent responses can hide numerical mistakes.

The implemented system is designed with that warning in mind. The LLM is not the source of calories or macros. Those values come from the nutrition database and mapping table. The recommendation layer receives already-computed values and uses them to generate advice. This makes the system more defensible for a nutrition-aware project, even though it requires more backend work.

2.2.18 Indian Food Classification and Dietary Recommendations Using CNNs

The edge CNN dietary recommendation paper uses deep CNN-based food classification and localized diet advice on Raspberry Pi-style hardware [18]. It shows that Indian food recognition can be connected to recommendation workflows outside a purely academic model benchmark.

The implementation differs in deployment style. Instead of requiring specialized edge hardware, it uses a web frontend, FastAPI backend, YOLO inference service,

SQLite nutrition database, and recommendation API. This makes the prototype easier to access from a normal browser while keeping the same broad goal: convert recognized Indian food into practical dietary feedback.

2.3 Comparative Summary

Table 2.1 gives a concise portrait-mode summary of the supplied papers. The detailed comments are kept in the paper-wise review above so that the table can remain readable even with all comparison columns shown.

Table 2.1: Portrait summary of the supplied literature

S. No	Technique & Reference	About Technique	Applications	Dataset Used	Results	Limitations
1	YOLOv5, YOLOv7 and YOLOv8, IndianFoodNet [1]	Single-stage detectors localize foods with bounding boxes, class labels, and confidence scores.	Indian food detection in meal images.	IFN: 5,500+ images, 30 classes.	YOLOv8 gave the strongest reported precision among tested models.	Detection is not connected to a nutrition database.
2	CNN, ViT and Con-vNeXT, Khana [2]	Deep classifiers and retrieval baselines model fine-grained Indian cuisine images.	Dish classification, segmentation, and retrieval.	Khana: about 131K images, 80 labels.	Provides a large Indian cuisine benchmark.	Mainly benchmark-oriented; not a complete nutrition workflow.
3	Segmentation and prototype matching, Indian Thali [3]	Segmentation separates visible thali components before classification and nutrition summarization.	Multi-item thali analysis.	Indian Thali Dataset and Weight Estimation Dataset.	Handles cluttered and overlapping thali foods.	Uses controlled capture hardware; less convenient for normal users.
4	SAM and SE-DenseNet121, single-item training [4]	SAM proposes food regions; SE-DenseNet121 classifies each segment.	Multi-dish recognition from single-item training data.	IFN30 and separate thali images.	97.48% single-item accuracy in the reported study.	Two-stage pipeline is heavier than direct YOLO detection.

S. No	Technique & Reference	About Technique	Applications	Dataset Used	Results	Limitations
5	Parameter-optimized CNN [5]	A lightweight CNN is tuned for lower inference cost and food calorie estimation.	Fast image-based calorie assistant.	Food image datasets used for regular meal recognition.	About 85% accuracy with real-time focus.	Mostly generic food coverage; weak Indian food specificity.
6	MobileNetV3 transfer learning [6]	Pretrained MobileNetV3 is fine-tuned with rotation and horizontal-flip augmentation.	Indian food image classification.	9 Indian food classes, about 2,700 images.	Accuracy improved to 95.3% with augmentation.	Very limited class diversity.
7	Indian Nutrient Databank [7]	Cooked-recipe nutrient values are built from ingredient data and recipe composition.	Reliable macro lookup for Indian recipes.	1,095 food items and 1,014 cooked recipes.	Creates an open Indian food composition resource.	Static database still needs class-name mapping.
8	Alias and fuzzy semantic mapping [8]	Name normalization, aliases, and partial matching link visual labels to nutrition rows.	Visual-to-nutrition dataset compatibility.	Visual food labels and nutrition tables.	Identifies the key mapping bottleneck.	Depends on clean names, aliases, and structured food taxonomy.
9	LLM and food knowledge graph, FKG.in [9]	LLMs help resolve regional recipe text and knowledge-graph nutrition entities.	Recipe composition analysis.	FKG.in, IFCT, INDB, and Nutritionix sources.	Supports automated nutrition aggregation from recipes.	Text-based; does not begin from a food photograph.
10	Barcode scanning and hybrid scoring, Label AI [10]	UPC barcode lookup retrieves packaged-food data and computes a health score.	Packaged food fitness planning.	Open Food Facts product records.	Produces a 0–10 nutrition-style score.	Cannot help home-cooked meals without barcodes.

S. No	Technique & Reference	About Technique	Applications	Dataset Used	Results	Limitations
11	Multi-task ML nutrition estimation [11]	Food ingredients are predicted from photos and then combined with database values.	Nutrition estimation from food photos.	Yelp food photos and nutrition datasets.	Reported 85% ingredient-attribute accuracy.	Ingredient mass and serving size remain uncertain.
12	Prompted LLM meal planning, NutriGen [12]	LLM prompts are constrained with reliable nutrition references and user preferences.	Personalized meal-plan generation.	Personalized nutrition database and USDA references.	Llama 3.1 and GPT-3.5 showed low calorie-target error.	Does not visually verify what the user ate.
13	OCR and LLM advice [13]	OCR extracts printed label text and an LLM generates personalized advice.	Packaged-food nutrition guidance.	Food product labels and user profiles.	Shows feasible CV-to-LLM advice.	Reads labels, not cooked food pixels.
14	DeepSeek-R1 and ChatGPT web planner [14]	Generative models process health inputs and produce meal plans.	Health-aware personalized diet planning.	User profiles and health constraints.	Reported stronger personalization than compared planners.	Recommendation quality depends on accurate user input.
15	YOLOv8, CNN and NLP, Diet Engine [15]	YOLOv8 detects food; NLP/chatbot modules provide dietary suggestions.	Real-time food nutrition assistant.	Custom dataset with 6,692 images.	Reports real-time classification and personalized guidance.	Computationally heavy and less focused on Indian food.
16	Flask, SQLAlchemy and diet rules, Smart Diet Planner [16]	A web backend stores foods, logs meals, and calculates user nutrition needs.	BMR, TDEE, diabetic filtering, and meal alternatives.	Indian food database with 500+ records.	95% calculation accuracy and 87% user satisfaction.	Depends mainly on manual food logging.

S. No	Technique & Reference	About Technique	Applications	Dataset Used	Results	Limitations
17	Vision-language nutrient estimation, ChatGPT evaluation [17]	ChatGPT identifies foods and estimates weight and nutrients from meal photos.	AI dietary assessment benchmark.	114 meal photographs.	93% food identification precision.	Poor nutrient agreement; exact macro values are unreliable.
18	Raspberry Pi and CNN recommendations [18]	Edge CNN classification links food recognition with diet suggestions.	Local meal analysis and diet recommendation.	Custom Indian meal images.	Gives immediate food and nutrition feedback.	Requires edge hardware and still faces portion-estimation limits.

2.4 Research Gap

Across the supplied papers, three strengths appear repeatedly. First, Indian food recognition benefits from domain-specific data because generic food categories do not capture regional dishes well. Second, nutrition systems become more trustworthy when they use structured food-composition data instead of free-form estimates. Third, personalized recommendation systems are more useful when they receive grounded inputs such as detected foods, macro totals, health goals, and dietary constraints.

The gap is in the careful connection between these parts. A detector can find food but does not know nutrition. A nutrition database stores macros but does not know what appears in a user’s image. A language model can explain and recommend, but it should not be treated as the source of exact nutrient values. The implemented project sits in this middle ground by combining YOLO-based detection, a verified class-to-nutrition mapping layer, a SQLite nutrition database, deterministic macro calculation, and a recommendation service that uses structured backend outputs.

Chapter 3

METHODOLOGY

3.1 Methodological Overview

The methodology follows the practical order in which the system was built. The dataset had to be made consistent before model integration could be trusted, and the nutrition database had to be checked before recommendation output could be meaningful. Only after these two parts were reasonably stable did the backend and frontend become useful for testing. The work begins with YOLO-format food datasets, moves through class normalization, split generation, train-only augmentation, detector integration, nutrition import, class-to-food mapping, and finally user-facing API and interface development. Figure 3.1 summarizes the main workflow.

This order was chosen because errors in the early stages propagate into the later stages. For example, if the same dish is stored under two class names, the detector learns them as different outputs. Similarly, if a YOLO class is mapped to a wrong nutrition row, the frontend can display a neat result that is still incorrect. The methodology therefore gives importance to data checking and mapping verification, not only to model training.

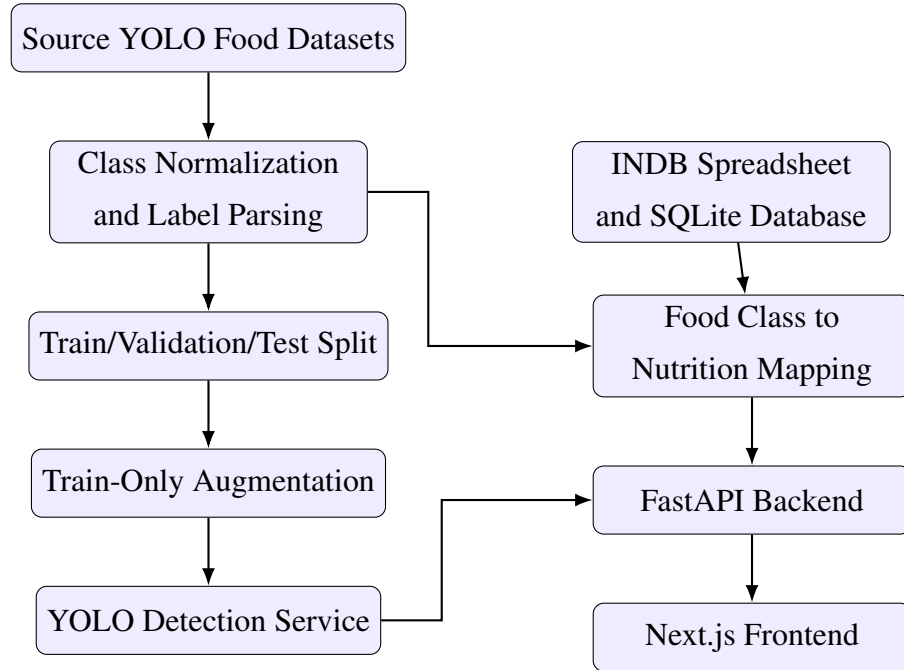


Figure 3.1: Methodological workflow for dataset, model, nutrition, backend, and frontend integration

3.2 YOLO Detection Approach

YOLO, short for You Only Look Once, is used in this project because it treats object detection as a single-pass prediction problem. For a food image, the model predicts bounding boxes, class probabilities, and confidence scores in one forward pass. This is suitable for a web application because the backend must return results quickly enough for a user to inspect the uploaded meal without a long delay.

The project uses the YOLO label format and YOLO-style inference workflow followed by recent Indian food detection systems such as IndianFoodNet and Diet Engine [1, 15]. During inference, the model returns candidate detections for visible food items. The backend then applies confidence filtering and non-maximum suppression so that duplicate boxes are removed and only the strongest detections are passed to nutrition lookup. Each final detection contains a class name, confidence value, and normalized bounding box.

The practical reason for using YOLO is not only speed. Indian meal photographs often contain more than one dish, so a whole-image classifier would usually miss part of the plate. YOLO allows the system to detect multiple foods in one image and map each detected class independently to the nutrition database. This gives the later macro calculation a clear trace: image region, detected class, database row, and final calories.

It also makes debugging easier because an incorrect macro value can be traced back to either the detected label or the mapping entry.

3.3 Source Dataset Collection

Five YOLO-format datasets were used as source datasets. Together they contained 40,797 source images across training, validation, and test folders before project-specific merging. Combining them increased food coverage, but it also introduced naming and annotation issues. Some labels were written in CamelCase, some used spaces, some used regional words, and some referred to the same dish with different spellings. These differences had to be resolved before training, otherwise the final model would contain duplicate or confusing class outputs. Table 3.1 summarizes the source data.

Table 3.1: Source datasets used for merged Indian food dataset

#	Source dataset	Classes	Train	Valid	Test
1	Indian food detection v1	17	7547	1053	453
2	Indian food v6	29	8088	213	105
3	Indian food v2 seven-class subset	7	1698	159	92
4	IndianFoodNet YOLO export	30	11387	1073	576
5	South Indian food detection v19	31	6478	1294	581

The raw class count exceeded the final class count because many labels referred to the same food. For example, `satham`, `rice`, and `WhiteRice` were normalized into `rice`. This step is not only cosmetic. Object detectors learn one output channel per class; if equivalent labels remain separate, the model is trained to treat the same food as different objects. The same normalized names were also useful later while building the nutrition mappings.

3.4 Canonical Class Normalization

The canonical label set contains 72 food classes. The normalization process converts raw source labels into snake-case names, resolves spelling differences, and merges regional variants. Manual checking was required for several labels because automatic normalization can clean punctuation and casing, but it cannot always decide whether

two regional names refer to the same dish. Table 3.2 shows examples.

Table 3.2: Examples of raw-to-canonical label normalization

Raw variants	Canonical class
AlooGobi, aloo gobhi, aloo_gobi	aloo_gobi
CoconutChutney, thengai chutney, coconut chutney	coconut_chutney
Idli, idly	idli
WhiteRice, rice, satham	rice
lemon satham	lemon_rice
medu vada, medu vada	medu_vada

3.5 YOLO Label Representation

Each YOLO label file contains one row per annotated object:

$$y = (c, x, y, w, h) \quad (3.1)$$

where c is the class identifier, x and y are normalized center coordinates, and w and h are normalized width and height. The normalized coordinate system makes labels independent of image resolution. During parsing, each raw class identifier is first resolved to its source dataset class name and then converted to the canonical class name. This two-step conversion was necessary because class identifiers are local to each dataset and cannot be compared directly across sources.

3.6 Dataset Splitting and Augmentation

The merged dataset uses a 70/15/15 target ratio for training, validation, and testing. Multi-label images are handled by building a multi-label representation of class presence. A stratified split is attempted so that classes are distributed across splits. If rare-label constraints make stratification infeasible, deterministic random splitting is used. The split stage was kept deterministic so that the same dataset can be regenerated and checked again if required.

Augmentation is applied only to the training split. This prevents synthetic transformations from leaking into validation or test evaluation. The augmentation strategy includes horizontal flipping and mild brightness and contrast adjustment. The purpose is deliberately modest: improve coverage for underrepresented classes without creating images that no longer look like realistic food photographs. More aggressive transformations were avoided because food colour and texture are important cues for distinguishing similar dishes.

Algorithm 1 Dataset merge and export procedure

```

1: Initialize empty sample list  $S$ 
2: for each source dataset  $D$  do
3:   Load source class names from data.yaml
4:   for each image-label pair in  $D$  do
5:     Parse YOLO labels
6:     Convert raw class names to canonical classes
7:     Add sample to  $S$ 
8:   end for
9: end for
10: Build multi-label matrix for all samples in  $S$ 
11: Split into train, validation, and test sets
12: Export images and labels using canonical class identifiers
13: Apply augmentation only to training samples below target count
14: Write final data.yaml, build log, and per-class count CSV

```

3.7 Nutrition Database Methodology

The nutrition database is built from the INDB spreadsheet. The required columns are `food_name`, `energy_kcal`, `protein_g`, `carb_g`, and `fat_g`. These are renamed to the runtime schema: `name`, `calories`, `protein_g`, `carbs_g`, and `fat_g`. A normalized food name column is added to support lookup and matching. Source metadata is also stored so that supplemental values can be distinguished from INDB rows.

The database contains two main tables. The first table, `indb_foods`, stores food records, macronutrients, and source information. The second table, `yolo_mappings`, stores precomputed mappings from YOLO classes to database food names. This separation keeps runtime lookup fast and makes the mapping decisions inspectable. It also allowed the mapping table to be regenerated during development without changing the

original nutrition records.

3.8 Nutrition Mapping Methodology

Food-to-nutrition mapping is performed conservatively. Manual semantic mappings are attempted first. These mappings are created by checking exact database food names and selecting the closest nutritionally meaningful record. Verified supplemental rows are used when the available INDB sheet has no safe entry for a dataset class. Fuzzy matching is kept as a fallback and used only when the score is sufficiently high. This matters because a similar string is not always a similar food, especially when sweets, curries, chutneys, and fried snacks share common words.

This mapping step took more time than initially expected. Several detector labels were easy for a person to understand but did not match the database wording exactly. For example, a regional or shortened class name could fail an exact lookup, while a fuzzy match could select a wrong but similarly spelled item. Because the final system displays numerical macro values, the mapping stage was treated as a validation problem rather than a convenience feature.

For a detected class c , the mapping function can be represented as:

$$M_c = \begin{cases} \text{manual}(c), & \text{if a verified manual mapping exists} \\ \text{supplemental}(c), & \text{if a verified supplemental row exists} \\ \text{fuzzy}(c), & \text{if } \text{score}(c) \geq 0.78 \\ \emptyset, & \text{otherwise} \end{cases} \quad (3.2)$$

The current mapping covers all 72 dataset classes. The classes `bhakarwadi`, `ghevar`, `jalebi`, `khandvi`, and `nandu_masala` were absent from the available INDB sheet, so they are handled through verified supplemental nutrition rows with source metadata rather than weak fuzzy substitutes.

3.9 Macro Calculation Methodology

The macro engine converts a daily calorie target into grams of carbohydrates, protein, and fat. Each goal defines a base macro split. Health conditions modify this split before normalization. This rule-based calculation was kept explicit so that the output can be checked manually during testing. If T is the target calorie value and p_c , p_p , and p_f are

the final percentages for carbohydrates, protein, and fat, then:

$$G_c = \frac{T \times p_c}{4 \times 100} \quad (3.3)$$

$$G_p = \frac{T \times p_p}{4 \times 100} \quad (3.4)$$

$$G_f = \frac{T \times p_f}{9 \times 100} \quad (3.5)$$

where G_c , G_p , and G_f are grams of carbohydrates, protein, and fat respectively. The values 4, 4, and 9 represent calories per gram for carbohydrates, protein, and fat. The generated values are later shown in the frontend as target macros against which the detected meal can be compared.

3.10 Validation Methodology

Validation is performed at multiple levels because a failure in any one layer can make the final output misleading. Dataset validation checks class names, split counts, and visual class samples. Database validation checks row counts and schema. Mapping validation checks class-to-food mappings, coverage, and source provenance. Service validation checks whether the backend can initialize the nutrition service and return expected lookup results. Frontend validation checks whether the user workflow can display detections, nutrition labels, and recommendations. This layered validation approach made debugging manageable because problems could be isolated to the dataset, database, service, or interface instead of being treated as one large application error.

Chapter 4

SYSTEM DESIGN

4.1 Design Goals

The system was designed around five practical goals: usability, modularity, traceable data flow, local deployability, and extensibility. Usability means that a user should be able to upload a food image and understand the result without knowing how the model works. Modularity keeps dataset building, model inference, nutrition lookup, recommendation generation, and interface rendering separate enough to debug independently. Traceable data flow makes detected labels, confidence scores, bounding boxes, and nutrition sources visible instead of hiding them inside the backend. Local deployability keeps the prototype easy to run on a developer machine. Extensibility leaves room for new classes, improved models, and better nutrition sources later.

These goals came from the way the project evolved. Early testing showed that a food-recognition result is difficult to trust if the label, confidence score, and nutrition source are not visible together. The design therefore avoids treating the model as a black box. Each layer returns structured information that can be checked during development and displayed clearly in the frontend.

4.2 High-Level Architecture

The project follows a layered architecture because each part of the system has a different responsibility. The data layer contains source datasets, the merged YOLO dataset, the INDB spreadsheet, and the SQLite nutrition database. The AI layer contains the YOLO model and inference service. The backend layer exposes APIs and coordinates the services. The frontend layer provides the user interaction. Figure 4.1 shows the high-level architecture.

This layering also reduced debugging time. When a wrong macro value appeared, it could be checked against the mapping table without retraining the detector. When the frontend display looked incorrect, the backend JSON response could be inspected separately. This separation made the project easier to complete as a full-stack application rather than as one large script.

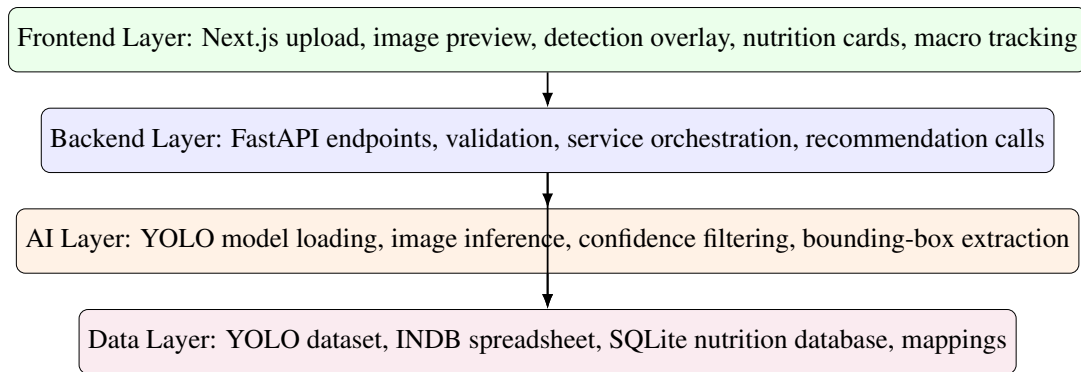


Figure 4.1: Layered architecture of the food recognition and nutrition system

4.3 Runtime Data Flow

The runtime flow begins when the user uploads an image. The frontend forwards the raw image to the backend. The backend validates the upload, sends it to the vision service, receives detected objects, and looks up nutrition for each detected label. The response includes image dimensions, bounding boxes, labels, confidence values, macro values, and nutrition source metadata. The frontend then renders the image with boxes and displays nutrition cards, so the user can see both the visual prediction and the nutrition consequence of that prediction.

The response format was kept detailed on purpose. During testing, it was not enough to know that the application had detected a food item. The team also needed to check where the box was drawn, how confident the model was, which database row was selected, and whether the macro values came from INDB or a supplemental source.

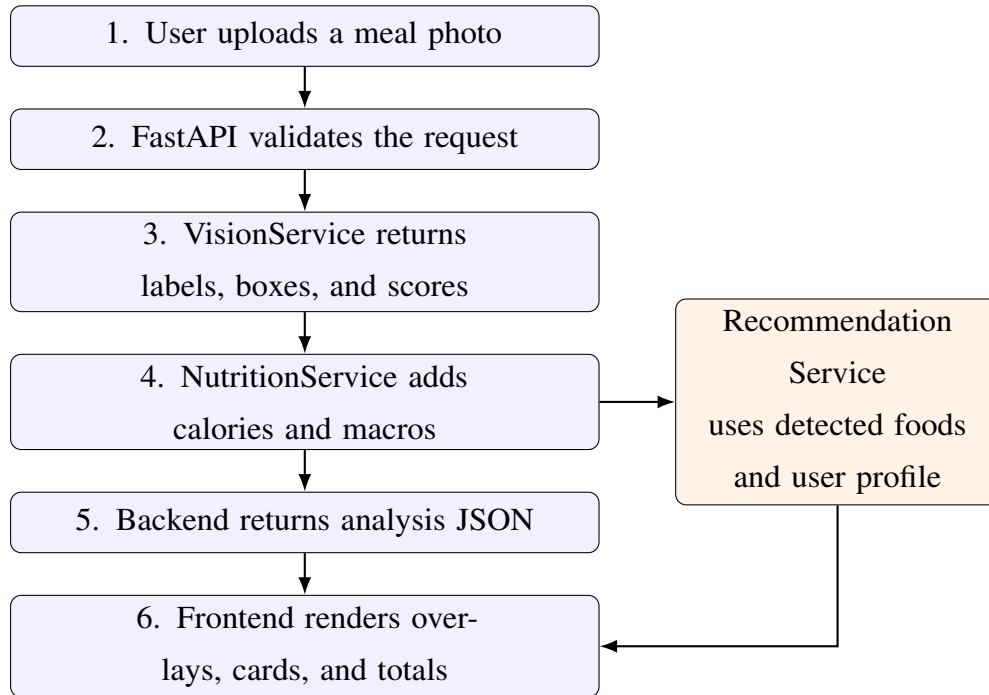


Figure 4.2: Runtime request-response flow

4.4 Backend Design

The backend is implemented with FastAPI, an ASGI-compatible Python framework suitable for typed request models, upload handling, and asynchronous service execution [19]. The backend contains the following core services:

- **VisionService:** loads the YOLO model and performs inference on uploaded images.
- **NutritionService:** connects to SQLite, caches food names, loads `yolo_mappings`, and returns macro values.
- **RecommendationService:** computes goal and health-condition context and generates dietary suggestions.

The backend design separates the endpoint layer from service logic. This makes it easier to test nutrition lookup without running image inference, and easier to test macro calculation without uploading files. This separation was useful when debugging the nutrition layer, because lookup errors could be reproduced using a single food name instead of repeating the complete image-upload workflow.

4.5 Frontend Design

The frontend is built with Next.js 14 and React [20]. It provides a single primary workflow: upload image, analyze, inspect detections, adjust serving estimates, review macro impact, and request recommendations. The frontend state is organized around the image analysis result, selected goal, target calories, selected health condition, and macro totals.

The user interface avoids a marketing-style landing page. The first screen is the actual tool: upload controls, user nutrition context, and result panels. This is appropriate because the application is an operational nutrition assistant, not a promotional website. During testing, the most useful interface behaviour was the ability to inspect the uploaded image and nutrition cards side by side, since this quickly showed whether the visual prediction and database output agreed with each other.

4.6 Database Design

The SQLite database contains a normalized nutrition table and a precomputed mapping table. Table 4.1 summarizes the database design.

Table 4.1: Runtime nutrition database tables

Table	Key columns	Purpose
<code>indb_foods</code>	<code>id</code> , food name, normalized name, calories, protein, carbohydrates, fat, source	Stores INDB and supplemental nutrition records.
<code>yolo_mappings</code>	YOLO class, matched food name, match score	Stores precomputed mappings from detector labels to database food rows.

The database is intentionally simple. A heavier database server would add setup work without improving the core prototype. Since the nutrition data is mostly read-only at runtime, SQLite is sufficient and keeps the project easy to run locally. It also makes the generated database file easy to inspect after rebuilding the mapping table.

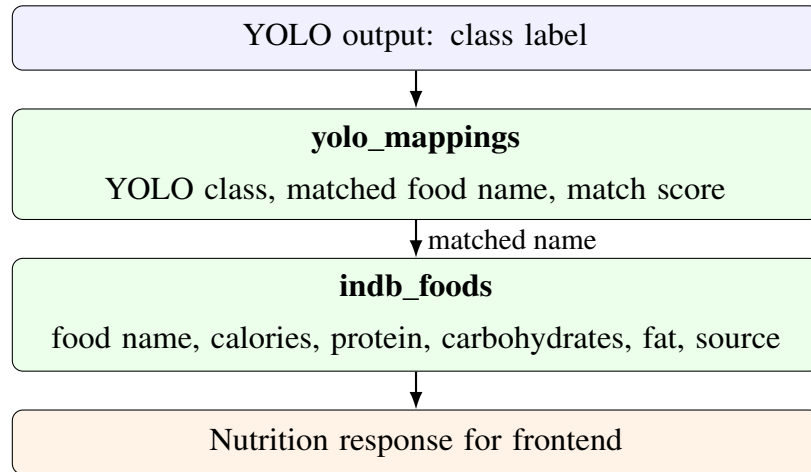


Figure 4.3: Entity relationship used for class-to-nutrition lookup

4.7 API Design

The API design exposes endpoints for health checks, image analysis, nutrition testing, nutrition validation, macro calculation, goal metadata, health-condition metadata, and recommendations. The additional testing and validation endpoints were kept because they helped verify the system during development and are useful for future maintenance. Table 4.2 lists the implemented endpoints.

Table 4.2: Implemented backend endpoints

Endpoint	Method	Purpose
/	GET	Root API description and endpoint list.
/api/health	GET	Returns service health and model-loaded status.
/api/analyze-food	POST	Accepts image upload, runs YOLO inference, and attaches nutrition data.
/api/test-nutrition/ <food_name>	GET	Debug endpoint for a single food nutrition lookup.
/api/validate-nutrition	GET	Validates nutrition lookup coverage for model classes.
/api/user/ calculate-macros	POST	Calculates goal-specific macro targets.
/api/user/goals	GET	Returns supported dietary goals.
/api/user/ health-conditions	GET	Returns supported health condition options.
/api/recommendations	POST	Generates dietary recommendations from detected foods and user profile.

4.8 Security and Validation Design

The system validates upload type, empty uploads, maximum file size, and service readiness. The backend enforces a maximum image upload size of 10 MB. Rate limiting is implemented for the food analysis endpoint to avoid repeated heavy inference calls. Secrets such as the Groq API key are loaded from environment variables and are not documented verbatim in the report.

These checks are basic, but they are important for a working prototype. Without them, a non-image file, an empty upload, or an unloaded model can produce unclear errors in the frontend. Returning structured failures makes the application easier to test and prevents the user interface from showing incomplete nutrition results.

4.9 Design Trade-Offs

The design uses per-100-gram nutrition values rather than automatic portion estimation. This choice limits exact calorie accuracy but improves reliability because monocular portion estimation requires plate scale, camera distance, depth, and food volume assumptions. The system instead provides serving multipliers in the frontend, allowing the user to adjust values interactively when the displayed serving is too small or too large.

The design also keeps explicit nutrition mappings in the database build process. This is less automatic than fuzzy matching, but it is safer for nutrition applications. Wrong nutrition values can be more harmful than missing nutrition values; therefore, classes without safe INDB rows are handled only through verified supplemental entries with recorded source metadata. This trade-off makes the prototype slightly more manual to maintain, but it gives better control over a health-related output.

Chapter 5

IMPLEMENTATION

5.1 Development Environment

The project is implemented as a full-stack application rather than a standalone notebook experiment. This mattered during development because the detector, nutrition database, API response format, and frontend display all had to agree with each other. A change in one part often affected another part; for example, a class-name change in the dataset also required checking the nutrition mapping and frontend display name. The backend uses Python, FastAPI, PyTorch, Ultralytics YOLO, Pillow, NumPy, Pandas, OpenPyXL, SQLite, and AioSQLite. The frontend uses Next.js, React, TypeScript, Tailwind CSS, and Lucide icons. The repository is organized into `backend/`, `frontend/`, `data/`, `models/`, `scripts/`, and `notebooks/` so that model assets, data processing, API code, and interface code remain separated.

5.2 Dataset Build Script

The dataset build script is located at `scripts/build_master_dataset.py`. It handles the repetitive work that would be error-prone if done manually: source dataset scanning, class-name loading, YOLO label parsing, canonical class conversion, multi-label split creation, image export, training augmentation, and artifact writing. It writes three key outputs: `data.yaml`, `build_log.txt`, and `split_class_counts_before_after.csv`.

This script was one of the most important parts of the implementation because the source datasets did not follow a single naming convention. Some files had class names embedded in filenames, some labels had to be read through dataset-specific `data.yaml` files, and several raw names referred to the same final dish. Automating the process reduced manual copying mistakes and made it possible to rebuild the dataset

after correcting normalization rules.

The final dataset contains 72 canonical classes. The generated balanced dataset contains 36,852 training image-label pairs, 5,922 validation pairs, and 5,919 test pairs. The total exported image-label pairs are 48,693. The larger secondary export retained in `data/master_dataset/` contains more aggressive train augmentation and is documented as an archived artifact.

5.3 Final YOLO Training Notebook

The final model training workflow is recorded in the final training notebook under `notebooks/`. The notebook is configured for Kaggle execution, loads the generated 72-class `data.yaml`, resumes the `yolo11s_indian_food` run from an existing checkpoint, and trains to 100 total epochs. The final resumed stage covers epochs 56 to 100 and uses YOLO11s with 640-pixel images, batch size 16, automatic mixed precision, checkpoint saving every 10 epochs, and plot generation enabled.

Training was resumed instead of restarted because the earlier checkpoint had already learned useful features from the merged dataset. The resumed run also made it possible to compare the later epochs through `results.csv` and training plots. The model was not selected only from the final epoch; the best exported weight was used because validation mAP peaked earlier than the last epoch.

The run used PyTorch 2.10.0 with CUDA 12.8 on a Tesla T4 GPU. The exported artifacts include `best.pt`, `last.pt`, checkpoint weights, training curves, precision-recall curves, confusion matrices, prediction batches, and `results.csv`. The best model file was also exported as `yolo11s_indian_food_best.pt`, which is the detector artifact used for final evaluation.

5.4 Visual Class Verification Implementation

To verify that class names correspond to actual images, a visual verification script was created at `scripts/verify_food_classes_with_images.py`. The script groups exported images by the canonical class suffix in the filename, samples four images per class, parses corresponding labels for object crops, and writes contact sheets. This step was useful because file counts alone cannot reveal whether a class name is visually believable. The original eight sheets were then split into sixteen half-sheets so the labels and sampled images remain readable in print. The full half-sheet set and the detailed annotation-issue table are placed in Appendix A to avoid repeating the same visual

material in the implementation chapter.

The visual review confirmed that all 72 classes have actual image samples and that the class-name list is aligned. It also exposed a few practical dataset issues, such as a crop that did not match the expected food or a box that was placed on the wrong side item. These were not class-order failures, but they are still important because object detectors learn from the exact boxes provided during training.

5.5 Nutrition Database Build Implementation

The nutrition database build script is located at `backend/scripts/merge_db.py`. It reads the INDB spreadsheet, verifies required columns, normalizes food names, removes duplicate normalized names, converts nutrition columns to numeric values, and rebuilds the local SQLite nutrition database. The script then loads the current dataset classes and appends legacy model labels so that older deployed labels still resolve correctly.

Manual mappings are applied before fuzzy matching. This prevents incorrect string-based matches. For example, `aloo_gobi` maps to Potato cauliflower (Aloo gobhi), `palak_paneer` maps to Spinach paneer (Palak paneer), and `rice` maps to Boiled rice (Uble chawal). The script also adds source-backed supplemental rows for classes that are present in the detector dataset but absent from the available INDB sheet. This approach required more manual checking, but it avoided several wrong matches found during early testing.

5.6 Vision Service Implementation

The vision service loads YOLO model weights and processes uploaded images. It converts input bytes into an image representation, performs inference, applies class-level confidence rules, extracts bounding boxes, and returns structured detection dictionaries. Each detection contains the food label, confidence score, bounding box, and image dimensions.

The model is warmed up during backend startup with a dummy image. This reduces the first real request latency, which otherwise can make the first upload feel unusually slow. The backend executes inference in a thread pool so that the asynchronous API loop is not blocked by model computation. Keeping the output as dictionaries also made it easier to attach nutrition data without changing the model inference code.

5.7 Nutrition Service Implementation

The nutrition service initializes by connecting to `data/nutrition.db`. It caches all food names and normalized names and loads the `yolo_mappings` table. Runtime lookup follows this order:

1. Check precomputed YOLO mapping.
2. Try exact normalized-name match.
3. Try partial normalized match.
4. Generate alias-based variations.
5. Use fuzzy matching as the last fallback.

The precomputed mapping step is the most important because it avoids incorrect fuzzy matches. The returned nutrition object includes display name, macro values, unit, source, and food-found status. During testing, this status field was useful because the frontend could distinguish between a detected food with nutrition data and a detected food whose nutrition record was missing or uncertain.

5.8 Recommendation and Macro Implementation

The macro calculation function accepts a target calorie value, a selected goal, and a health condition. It applies base goal splits and health-condition modifiers, normalizes the result, and converts calories into grams. The recommendation service uses detected foods and user context to generate three practical suggestions. The prompt includes detected foods, calories, goal, health profile, and health-specific dietary guidance. The calculation was kept separate from the recommendation text so that macro targets can be tested even when the external recommendation service is not configured.

5.9 Frontend Implementation

The frontend is implemented in `frontend/app/page.tsx` and reusable components. `ImageUpload` manages drag-and-drop upload and file preview. `ResultsPanel` displays detection cards, nutrition values, serving controls, macro totals, and recommendations. `NutritionLabel` displays calories, protein, carbohydrates, and fat. `ImageOverlay` draws bounding boxes over the uploaded image.

The serving controls were included because the detector can identify a food item but cannot know its exact mass from a single image. This kept the interface honest: the system provides a calculated baseline, and the user can adjust the assumed serving when the visible quantity is clearly larger or smaller.



Figure 5.1: Frontend detection overlay showing Bhatura and Chole detections

5.10 Code Organization

Table 5.1 summarizes the important implementation files.

Table 5.1: Important implementation files

File or component	Purpose
<code>backend/main.py</code>	FastAPI application, endpoints, startup sequence, request validation.
<code>vision_service.py</code>	YOLO model loading, inference, and detection formatting.
<code>nutrition_service.py</code>	SQLite lookup, mapping cache, and nutrition response creation.
<code>recommendation service</code>	Goal-aware and health-condition-aware recommendations.
<code>merge_db.py</code>	INDB import, supplemental nutrition rows, and YOLO-to-database mapping creation.
<code>food_normalizer.py</code>	Food-name normalization and variation generation.
<code>dataset build script</code>	Merged dataset construction and augmentation.
<code>Class verification script</code>	Contact-sheet generation for visual class verification.
<code>page.tsx</code> and UI components	Main frontend workflow, detection cards, serving controls, and macro display.

5.11 Implementation Challenges

The most significant challenges were label inconsistency, nutrition mapping ambiguity, and multi-component integration. Label inconsistency was addressed through canonical class naming. Nutrition ambiguity was addressed by explicit mappings and conservative fallback. Integration complexity was handled by separating services and keeping JSON response shapes structured.

In practice, the mapping work took more attention than expected because a visually correct food label can still produce the wrong macro values if it is matched to the wrong database row. Another challenge was keeping the training artifacts, deployed model name, dataset classes, and frontend labels synchronized. Small mismatches did not always cause immediate errors, but they could produce confusing results during testing. For this reason, validation scripts and debug endpoints were retained in the project instead of being removed after the first successful run.

Chapter 6

EXPERIMENTS AND VALIDATION

6.1 Experimental Scope

The experiments in this project are based on evidence that could be verified from the repository and generated artifacts. The evaluation covers dataset construction, YOLO11s detector training, validation and test metrics, class-name verification, nutrition database integrity, mapping coverage, backend service behavior, frontend workflow, and qualitative detection output. The final training notebook and exported result directory provide the detector metrics used for reporting quantitative mAP, precision, recall, confusion-matrix, and prediction-batch evidence for the latest 72-class model.

The validation work was not limited to checking whether the model produced a number. Several parts of the system had to be checked together. For example, a detection result was useful only if the class existed in the final `data.yaml`, the nutrition service could resolve the class name, and the frontend could display the returned macro fields correctly.

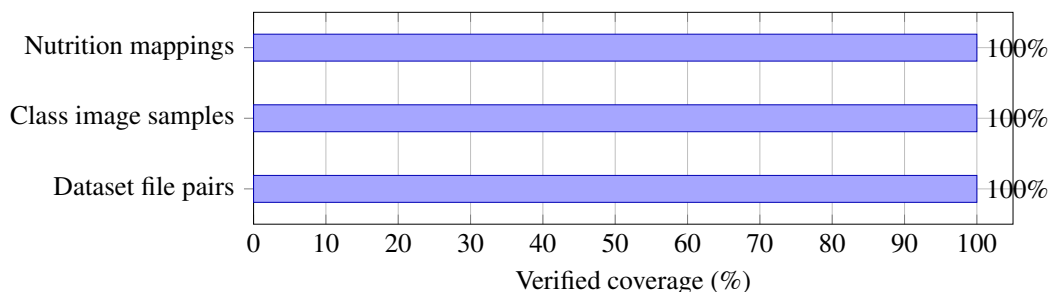


Figure 6.1: Verified system-level coverage used for validation

6.2 Dataset Validation

The first validation task checks whether the merged dataset exists, contains expected split folders, and has a complete 72-class data.yaml. This was a necessary first step because a missing label file or a class-count mismatch would make later training results unreliable. The generated dataset summary is shown in Table 6.1.

Table 6.1: Generated dataset summary

Property	Value
Canonical classes	72
Train images	36,852
Train labels	36,852
Validation images	5,922
Validation labels	5,922
Test images	5,919
Test labels	5,919
Total exported image-label pairs	48,693
Split target ratio	70/15/15

The split counts indicate that the generated dataset is complete: every exported image has a corresponding label file. Validation and test augmentation are disabled so that held-out samples remain real images rather than transformed versions of the training data. This makes the reported validation and test metrics more meaningful because the model is not being evaluated on augmented copies of training-style images.

6.3 YOLO Training and Detector Evaluation

The final detector experiment used YOLO11s on the 72-class dataset. The run resumed from a previous checkpoint at epoch 56 and continued to 100 total epochs. The resumed training approach was chosen because the earlier checkpoint was already stable enough to continue, and restarting from the beginning would have increased training time without adding new data. Table 6.2 summarizes the final training setup recorded in the final training notebook and the exported args.yaml.

Table 6.2: Final YOLO11s training setup

Property	Value
Model	YOLO11s object detector
Classes	72
Image size	640 pixels
Batch size	16
Training stage	Resumed from epoch 56 to epoch 100
Training time for resumed stage	10.636 hours
Hardware	Tesla T4 GPU, 15.64 GB VRAM
Framework	Ultralytics 8.4.60, PyTorch 2.10.0, CUDA 12.8
Best exported weight	yolo11s_indian_food_best.pt

The resumed training stage steadily reduced training and validation losses. The highest validation mAP@50 in `results.csv` was 0.83379 at epoch 78, while the final epoch recorded precision 0.8528, recall 0.8042, mAP@50 0.82806, and mAP@50:95 0.69164. The fact that the best validation value occurred before the final epoch is normal in detector training and supports using the exported best checkpoint for evaluation. Figure 6.2 and Figure 6.3 show the training curves generated from the final run.

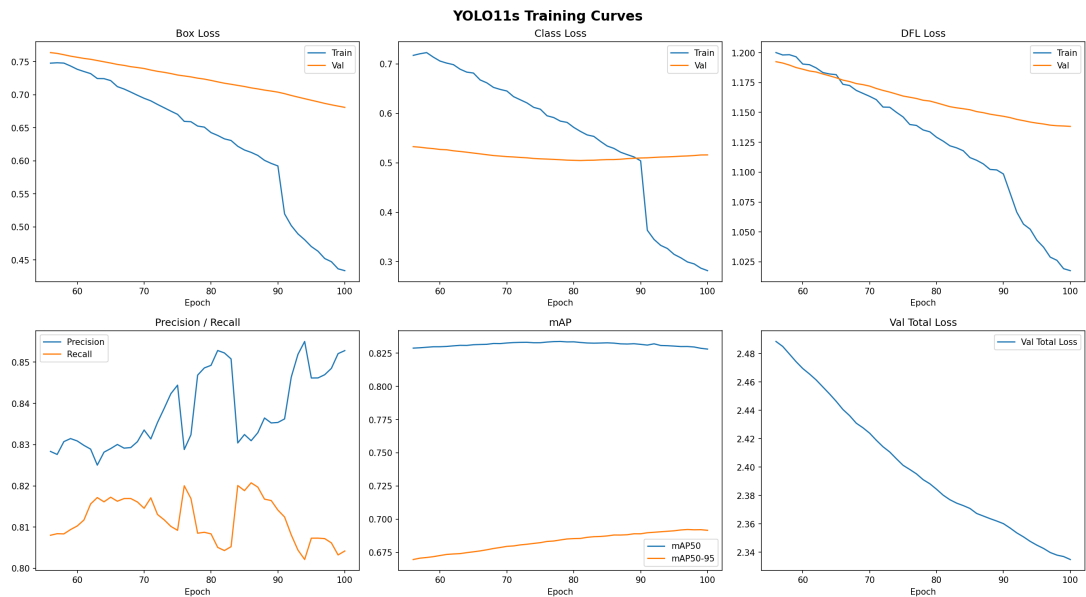


Figure 6.2: YOLO11s final training curves from the resumed 72-class run

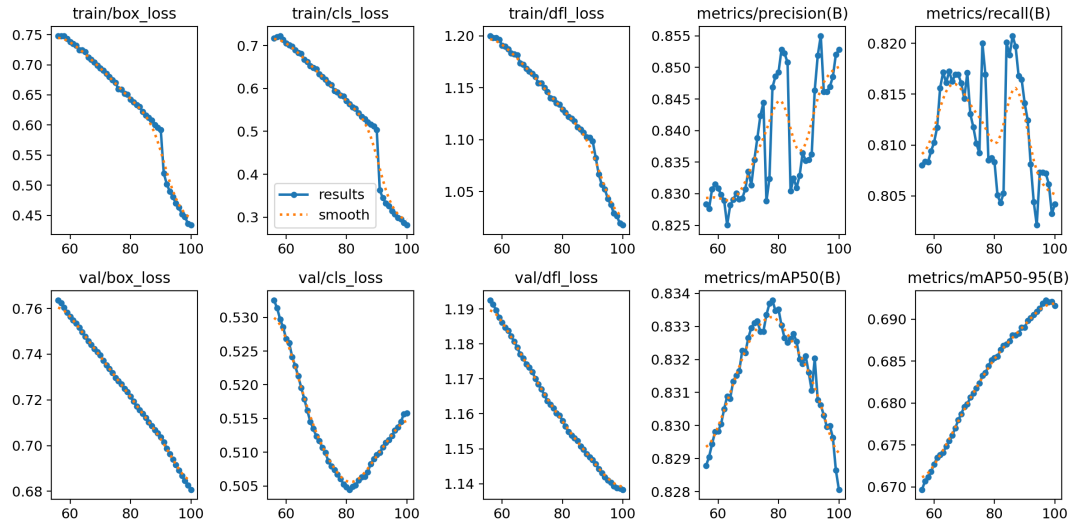


Figure 6.3: Ultralytics result plot for the final YOLO11s run

The best exported model was evaluated on both validation and held-out test splits. Table 6.3 reports the aggregate detector metrics.

Table 6.3: Final YOLO11s aggregate detector metrics

Split	Images	Instances	Precision	Recall	mAP@50	mAP@50:95
Validation	5,922	8,812	0.847	0.807	0.830	0.692
Test	5,919	8,875	0.842	0.801	0.820	0.680

The validation run reported mAP@75 of 0.765, and the test run reported mAP@75 of 0.751. Both validation and test evaluation reported approximately 11.0 ms inference time per image on the Kaggle Tesla T4 environment, excluding application-level upload and response overhead. The reported inference time therefore describes the model stage only; the full web application also includes file upload, image decoding, nutrition lookup, and frontend rendering.

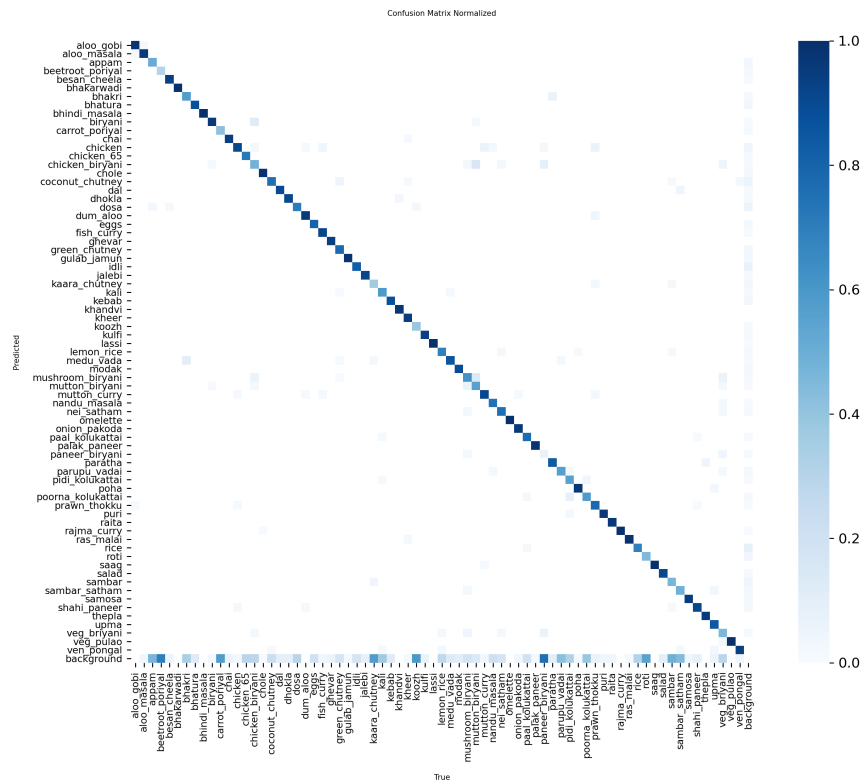


Figure 6.4: Normalized confusion matrix for the YOLO11s held-out test evaluation

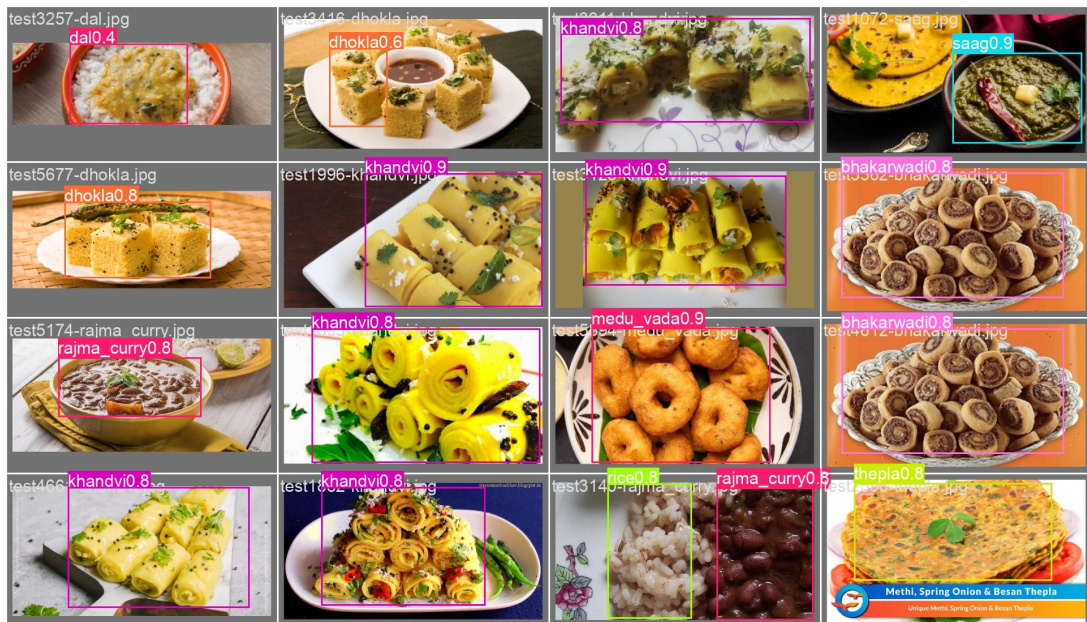


Figure 6.5: Sample YOLO11s predictions from the held-out test split

The aggregate metrics show that the final detector generalizes from validation to the held-out test split with a small drop in mAP. The normalized confusion matrix still shows uneven class behavior, which is expected in a 72-class food detector where several

dishes are visually similar and some classes contain fewer unique source images. In visual inspection, most mistakes were easier to understand when the foods shared colour, texture, or serving style, such as rice-based dishes, paneer gravies, and chutney-side combinations.

6.4 Class Verification Experiment

The class verification experiment generated contact sheets with four images per class. The goal was to catch practical mistakes that are easy to miss in code: class-order shifts, naming mismatches, or labels that do not match the visible food. The script found image samples for all 72 classes. Visual inspection found no class-list order problem. The issues found were sample-level annotation issues rather than class-list errors.

This experiment was useful because it gave a fast human-readable check of the dataset. Looking only at CSV counts would not show whether a jalebi sample actually looked like jalebi or whether a chutney box was placed on the correct food region. The contact sheets made these issues visible without opening hundreds of files manually.

Table 6.4: Visual class verification result

Check	Result
Classes checked	72
Classes with image samples	72
Classes without image samples	0
Contact sheets generated	8
Displayed samples per class	4
Class-order problem found	No
Sample-level issues found	3

6.5 Nutrition Database Validation

The nutrition database validation checks schema and row counts. The database contains 1,010 records in `indb_foods` and 103 records in `yolo_mappings`. The food table is based on INDB and five source-backed supplemental rows for classes that were absent

from INDB. The mapping count is larger than 72 because the table includes the expanded dataset classes and legacy deployed-model labels for backward compatibility.

The row-count check was followed by lookup tests because a database can contain records but still fail to return the correct food for a detector label. This was especially important after the mapping logic was changed from mostly fuzzy matching to explicit class-to-food mappings.

Table 6.5: Nutrition database validation result

Table	Purpose	Count
indb_foods	INDB plus supplemental nutrition records	1,010
yolo_mappings	YOLO class to database mappings	103

6.6 Nutrition Mapping Coverage Experiment

The mapping coverage experiment checks how many of the 72 dataset classes have a safe nutrition mapping. The result is shown in Table 6.6. Coverage was measured on the final canonical class list rather than only on the currently detected labels, because the model can potentially return any of the 72 classes during use.

Table 6.6: Nutrition mapping coverage for 72-class dataset

Metric	Value
Total dataset classes	72
Mapped classes	72
Unmapped classes	0
Mapping coverage	100.00%

The classes `bhakarwadi`, `ghevar`, `jalebi`, `khandvi`, and `nandu_masala` were not present as safe INDB rows, so verified supplemental entries were added with source metadata. The supplemental values use a FatSecret product nutrition label for `bhakarwadi` and Clearcals recipe nutrition pages for the remaining dishes [21–25]. This keeps the full 72-class dataset usable while avoiding unsafe fuzzy substitutions. These rows are not treated as equal to primary INDB coverage in the discussion; they are documented as supplemental values.

Table 6.7: Verified supplemental nutrition rows

Class	Database row	kcal	Protein	Carbs	Fat
bhakarwadi	Bhakarwadi	510.0	10.76	57.03	26.55
ghevar	Ghevar	351.2	3.5	39.6	19.9
jalebi	Jalebi	316.8	3.4	44.6	13.8
khandvi	Khandvi	232.7	9.3	21.7	12.1
nandu_masala	Nandu Kari (Crab masala)	128.1	7.0	7.0	8.0

6.7 Service Lookup Validation

Service-level lookup validation was performed for classes that previously suffered from incorrect fuzzy matches. These examples were selected because they represented real failures observed during development, not only random test cases. The corrected results are shown in Table 6.8.

Table 6.8: Corrected nutrition lookup examples

Input class	Returned INDB display name
aloo_gobi	Potato cauliflower (Aloo gobhi)
palak_paneer	Spinach paneer (Palak paneer)
rice	Boiled rice (Uble chawal)
AlooGobi	Potato cauliflower (Aloo gobhi)
WhiteRice	Boiled rice (Uble chawal)

This confirms that the revised database mapping handles both canonical dataset labels and legacy deployed-model labels. It also confirms that the nutrition service can resolve common casing and naming differences such as AlooGobi and WhiteRice.

6.8 Backend API Validation

The backend validation focuses on service readiness and response structure. The analysis endpoint returns HTTP 503 if vision or nutrition services are unavailable. Empty

images, non-image MIME types, and uploads exceeding 10 MB are rejected. The analysis response includes food label, confidence, bounding box, nutrition source, macro values, and food-not-found status. This structured response is important because the frontend depends on predictable fields for visualization and macro tracking. It also makes debugging easier when a detection is correct but the nutrition object is missing or incomplete.

6.9 Frontend Workflow Validation

The frontend workflow was validated qualitatively by uploading sample food images and verifying that the interface displays bounding boxes, detected labels, confidence values, nutrition cards, serving controls, and recommendation text. Figure 5.1 in Chapter 5 shows a successful example with Bhatura and Chole detections. The interface also handles no-food responses and backend error messages.

The qualitative workflow test was important because some issues only appear after the model response reaches the browser. For example, box coordinates must align with the displayed image size, serving changes must update macro totals, and recommendation text must use the current user goal rather than a previous state.

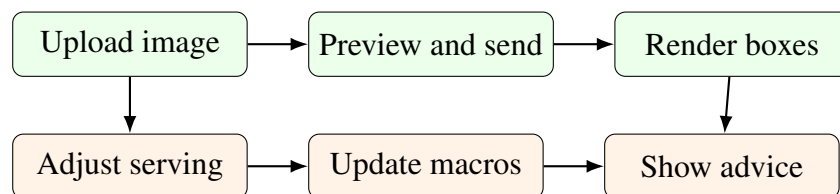


Figure 6.6: Frontend workflow checks used during qualitative validation

6.10 Macro Calculation Validation

Macro validation checks whether a target calorie value is converted correctly to grams. For example, for a 2,000 kcal weight-loss goal with a 40/35/25 split, the carbohydrate target is 200 g, protein target is 175 g, and fat target is approximately 56 g. Health profiles modify these base splits. The formula was discussed in Chapter 3. These calculations are simple enough to verify manually, which is useful because they directly affect the recommendation context shown to the user.

6.11 Experimental Findings

The experiments show that the dataset and application pipeline are internally consistent. The final YOLO11s model achieved 0.830 mAP@50 and 0.692 mAP@50:95 on validation, and 0.820 mAP@50 and 0.680 mAP@50:95 on the held-out test split. The system can detect food classes, map all 72 expanded dataset classes to nutrition records, and display results in the frontend. The results are strong enough for a final-year prototype, while still showing practical limits such as class confusion among similar foods, dependence on per-100g nutrition values, and the difference between model inference time and full application response time.

Chapter 7

RESULTS

7.1 Dataset Engineering Results

The dataset engineering phase produced a 72-class merged dataset. The important result was not just a larger dataset, but a cleaner label space built from five inconsistent sources. This mattered during later stages because the same canonical class names were used in the YOLO training file, nutrition mapping table, backend response, and frontend display. Table 7.1 summarizes the types of classes represented in the final dataset.

Table 7.1: Representative categories in the 72-class dataset

Category	Example classes
Rice dishes	rice, lemon_rice, sambar_satham, veg_briyani, veg_pulao, ven_pongali
Flatbreads and fried breads	roti, paratha, bhakri, bhatura, puri, thepla
Curries and gravies	chole, rajma_curry, fish_curry, mutton_curry, shahi_paneer, palak_paneer
Snacks	samosa, onion_pakoda, medu_vada, parupu_vadai, bhakarwadi, khandvi
Sweets and desserts	gulab_jamun, ras_malai, kheer, kulfi, modak, jalebi
Chutneys and sides	coconut_chutney, green_chutney, kaara_chutney, raita, salad

The class verification output confirms that all classes have image samples. This matters because a dataset configuration can list classes even when no usable labels exist

for them. The verification step confirmed that this is not the case here. It also showed that the dataset is not perfect, since a few individual annotations were found to be noisy. For a project prototype, this was acceptable, but it is still useful to record because model performance depends strongly on annotation quality.

7.2 Detector Training Results

The final YOLO11s training run produced a usable 72-class detector. The model was trained through epoch 100, with the final resumed stage covering 45 epochs from epoch 56 onward. During that stage, the best validation mAP@50 recorded in `results.csv` was 0.83379 at epoch 78. The final epoch recorded precision 0.8528, recall 0.8042, mAP@50 0.82806, and mAP@50:95 0.69164.

The exported best model was then evaluated separately on validation and held-out test splits. On validation, it achieved 0.847 precision, 0.807 recall, 0.830 mAP@50, and 0.692 mAP@50:95. On the held-out test split, it achieved 0.842 precision, 0.801 recall, 0.820 mAP@50, and 0.680 mAP@50:95. The small difference between validation and test mAP indicates that the model is not only fitting the validation split. At the same time, the confusion matrix shows that some dishes remain harder to separate, especially when they share colour, texture, or serving style.

7.3 Nutrition Mapping Results

The nutrition mapping results improved substantially compared with the earlier deployed-model-only mapping. Previously, fuzzy matching produced incorrect mappings such as `aloo_gobi` to a cauliflower dessert and `palak_paneer` to a generic cottage-cheese dish. These errors were important because the final output still looked numerical and confident even when the food record was wrong. The revised strategy maps these classes to more appropriate database entries and records source information for supplemental rows.

Table 7.2: Examples of improved nutrition mappings

Class	Corrected mapping	Reason
aloo_gobi	Potato cauliflower (Aloo gobhi)	Direct dish-level semantic match.
palak_paneer	Spinach paneer (Palak paneer)	Direct dish-level semantic match.
rice	Boiled rice (Uble chawal)	Best base record for plain rice.
sambar	Sambar	Exact database match.
shahi_paneer	Shahi paneer	Exact database match.

Some mappings remain approximate because the available INDB sheet does not contain every exact dish name. For example, `ven_pongali` maps to `Plain khitchdi`, and `chicken_biryani` maps to `Chicken pulao`. These approximations are documented through confidence scores and macro verification reports rather than hidden from the user. This is important because a prototype should make its assumptions visible, especially when the output is related to diet and health.

7.4 Application Results

The frontend displays detected foods with bounding boxes and nutrition cards. In the demonstrated Bhatura and Chole image, the detector produced boxes around the large bhatura and the chole bowl. The frontend used these boxes for overlays and used the backend response to show macro values. This confirms the practical connection from image upload to visual output, nutrition lookup, and user-facing guidance.

The application result is significant because it checks more than model accuracy. It verifies that the image can move through the upload flow, the backend can return structured detections, the nutrition service can attach macro values, and the frontend can render the combined result in a readable form.

7.5 Macro and Recommendation Results

The macro engine returns target grams for each dietary goal and health condition. For a fixed calorie target, different goals produce different distributions. A weight-loss goal

emphasizes protein more strongly, while an endurance goal emphasizes carbohydrates. Health profiles further adjust this split. This gives the recommendation system more context than a generic food recognition application.

The recommendation service produces three practical dietary suggestions based on detected foods and user profile. For example, if the detected meal is high in fried carbohydrates and the user chooses weight loss, the recommendation can advise smaller portions, additional protein, and lower-oil alternatives. If the user selects a diabetic profile, the advice can focus on carbohydrate moderation, fiber, and pairing with protein. The recommendation output was intentionally kept short because long diet explanations are less useful in a meal-logging workflow.

7.6 Result Synthesis

Taken together, the results show that the project is more than a detector experiment. The dataset, model, nutrition database, backend, and frontend all connect into a working pipeline. The most important outcome is that the visual detector output is traceable to explicit nutrition records and user-facing macro guidance. The results also show the boundaries of the prototype: portion size is still controlled by user adjustment, some food classes are naturally difficult to distinguish, and nutrition mappings depend on the availability of suitable food-composition records.

Chapter 8

DISCUSSION

8.1 Interpretation of Results

The results show that the useful part of a food recognition system is not the detector by itself. The detector tells the system what appears in the image, but the nutrition layer decides what that label means in calories and macros. The frontend then makes those numbers adjustable and understandable. The main outcome of this project is therefore the integration of these layers, not only the presence of a YOLO model.

The dataset engineering work was equally important. A model trained on inconsistent labels would produce confusing outputs even if the training process completed successfully. Canonical class normalization gave the detector a stable label space, and it also made nutrition mapping possible. The visual verification sheets were useful because they exposed real dataset issues that a file-count check would miss. A few noisy annotations remain, but the class list itself is aligned.

One practical observation from the project is that food recognition errors are not always obvious from metrics alone. A small class-label mistake can become more serious when it leads to a wrong nutrition record. For this reason, the discussion of results has to consider the complete chain from image to recommendation, not only the detector score.

8.2 Strengths of the System

The first strength is full-stack completeness. Many student and research prototypes stop after model inference. This project goes further by including dataset building, backend APIs, nutrition database integration, recommendation logic, and a frontend workflow that can actually be used and inspected. This made the work more difficult, but it also

made the final output more realistic.

The second strength is domain specificity. The dataset focuses on Indian foods, and the nutrition database uses Indian food names and macro values. The alias and mapping logic also accounts for regional naming patterns, which matters for labels such as `satham`, `medu_vadai`, `thengai_chutney`, and `nandu_masala`.

The third strength is conservative nutrition mapping. Classes are mapped through explicit semantic rules first, and classes absent from INDB are covered only when a source-backed supplemental row is available. This is important for health-related applications because weak fuzzy matches can produce misleading macro values.

The fourth strength is documentation and reproducibility. The project contains scripts for dataset construction, nutrition database creation, visual class verification, and macro verification. These scripts make the data pipeline inspectable instead of leaving the dataset and mappings as unexplained artifacts. This is useful for a final-year project because the work can be checked from generated files rather than only from screenshots.

8.3 Limitations

The most important limitation is portion size. The system reports macros per 100 grams and allows serving adjustment, but it does not estimate exact food mass from the image. This limits calorie precision. Accurate portion estimation would require depth information, reference objects, segmentation masks, or learned volume estimation.

The second limitation is nutrition source consistency. Five classes are covered through supplemental external sources because the available INDB sheet lacks safe equivalent entries. Some other classes use documented substitutes rather than exact dish rows. That is acceptable for a prototype, but a health-sensitive deployment would need stronger food-composition validation and expert review.

The third limitation is dataset noise. Visual verification found a small number of annotation issues. Large food datasets often contain such noise, but object detectors are sensitive to label quality. In this project, the noisy samples did not prevent a working detector from being trained, but they should still be treated as a limitation while interpreting the results.

The fourth limitation is uneven detector behaviour across visually similar dishes. The final YOLO11s metrics are strong enough for a working prototype, but the confusion matrix shows that some classes remain harder than others. This is especially important for a nutrition application because a wrong food class can propagate into an incorrect nutrition lookup even when the interface and database are functioning correctly.

Another limitation is that the recommendation layer depends on an external language model service when configured. This is acceptable for a prototype, but it makes the recommendation output harder to validate than the rule-based macro calculation. A self-trained recommendation model with fixed input-output constraints would provide better control in a later version.

8.4 Ethical and Practical Considerations

Nutrition applications can influence user behavior. Therefore, the system should avoid presenting estimates as medical advice. The recommendation text should be framed as general guidance, and users with medical conditions should consult qualified professionals. The application should also communicate uncertainty when a detected food uses supplemental nutrition data or when the model confidence is low.

Data privacy is another consideration. Food images can reveal personal habits, location context, and health patterns. A production version should minimize image retention, protect API communication, and provide clear user consent. The current prototype is local and does not implement cloud storage for uploaded images.

There is also a practical responsibility in how results are displayed. A bounding box and macro card can make an output look certain even when the food class or serving size is uncertain. For this reason, confidence scores, nutrition sources, and serving controls are important parts of the user interface.

8.5 Why Supplemental Rows Were Added

The classes `bhakarwadi`, `ghevar`, `jalebi`, `khandvi`, and `nandu_masala` were absent from the available INDB file. Mapping them through fuzzy matching would be unsafe because a sweet fried dessert should not be mapped to a generic sweet if the macro profile is different, and a crab masala class should not be mapped to a spice mix or unrelated seafood item. The database therefore adds verified supplemental rows for these classes and records their source URLs. This gives complete dataset coverage while keeping provenance visible.

This decision reflects a general rule followed during the project: it is better to document an approximate or supplemental value than to hide a weak match behind a confident interface. In a nutrition application, transparency is more important than making every lookup appear equally reliable.

8.6 Lessons Learned

The project shows that applied AI systems require more than model selection. Dataset curation, naming conventions, data contracts, API design, error handling, and UI workflow affected the final system as much as the detector choice. Food recognition is especially dependent on domain knowledge because visual labels and nutrition records rarely use identical names.

Another lesson is that validation artifacts are valuable. The class verification sheets made it possible to check 72 classes quickly and gave a simple way to discuss dataset quality without opening hundreds of files manually. Without such artifacts, label errors could remain hidden until model performance degrades.

The most practical lesson was that every stage needs a way to be checked independently. The dataset script, nutrition validation endpoint, macro reference tables, and frontend workflow checks were not extra polish; they were necessary to understand where a wrong output was coming from. This made the final report more evidence-based and made the system easier to explain.

Chapter 9

CONCLUSION

This project presented a full-stack AI-based food recognition system with nutrition-aware recommendations for Indian food. The system brings together a YOLO-based detector, a curated 72-class dataset, a SQLite nutrition database, a FastAPI backend, and a Next.js frontend. In the implemented workflow, a user uploads a food image, views detected foods with bounding boxes, checks macro values, adjusts serving assumptions, and receives dietary suggestions based on personal goals and health conditions.

The main objectives were achieved. Five YOLO-format source datasets were merged into a canonical Indian food dataset. The build process normalized raw labels, exported balanced splits, and produced machine-readable artifacts. Visual class verification confirmed that all 72 class names have corresponding image samples and that the class list is aligned. This step was important because the dataset had to support not only model training but also nutrition mapping and frontend display. The nutrition pipeline produced 1,010 food records, including five verified supplemental rows, and created 103 total YOLO-to-database mappings. For the expanded dataset, all 72 classes now have nutrition mappings.

The final YOLO11s detector was trained and evaluated on the expanded dataset. The best exported model achieved 0.830 mAP@50 and 0.692 mAP@50:95 on validation, and 0.820 mAP@50 and 0.680 mAP@50:95 on the held-out test split. The backend and frontend were implemented as a working local prototype rather than only a model experiment. The backend exposes image analysis, nutrition validation, macro calculation, goal, health-condition, and recommendation endpoints. The frontend provides image upload, detection visualization, macro display, serving adjustment, and recommendation review.

The work demonstrates that food recognition becomes more valuable when it is connected to nutrition databases and user-aware recommendation logic. It also shows that

dataset normalization and nutrition mapping are not minor details in Indian food systems; they are central to whether the output is trustworthy. The prototype still has limitations in portion estimation, annotation quality, class confusion among similar dishes, and nutrition source consistency. Even with these limitations, the project provides a working foundation for a more controlled recommendation model, better portion handling, mobile use, long-term tracking, and stronger safety review.

Chapter 10

FUTURE WORK

10.1 Self-Trained Recommendation Model

The current prototype uses a language model to generate natural-language dietary suggestions from detected foods, macro values, goals, and health conditions. A future version should replace this dependency with a self-trained recommendation model. The model can be trained on curated Indian meal patterns, nutrition rules, health-condition constraints, and expert-reviewed recommendation examples. This would make the recommendation layer more controllable, easier to validate, and less dependent on external LLM services.

The self-trained model should receive structured inputs from the existing backend, including detected food classes, calories, protein, carbohydrates, fat, serving estimates, user goal, calorie target, and health profile. Its output should be constrained to safe dietary suggestions and should include confidence or rule-trace information so that users can understand why a recommendation was produced. This would also make testing easier because the same input profile should produce consistent and reviewable advice.

10.2 Portion Size Estimation

The current system reports nutrition per 100 grams and allows serving adjustment. Future versions should estimate portion size using reference objects, depth estimation, segmentation masks, or plate-size assumptions. Instance segmentation could improve food-region estimation compared with bounding boxes, especially for rice, chutneys, and mixed curries. This enhancement would directly improve calorie usefulness because serving size is one of the largest sources of uncertainty in image-based food logging.

10.3 Mobile Deployment

A mobile version would make the system more practical for real meal logging. The front-end could be adapted as a progressive web app or native mobile application. Depending on device constraints, the model could run through server-side inference, quantized local inference, or a hybrid approach. Camera access, offline behaviour, and response time would need to be tested carefully because users are more likely to log food from a phone than from a desktop system.

10.4 Personalized Long-Term Tracking

Future versions can add user accounts, meal history, daily calorie budgets, trend charts, and weekly nutrition summaries. This would move the prototype from a single-image analyzer toward a long-term dietary assistant. The design should still allow users to edit detected meals manually, because long-term tracking becomes useful only when users can correct mistakes and maintain accurate records.

10.5 Clinical Safety and Explainability

Before deployment for health-sensitive users, the system should include stronger disclaimers, uncertainty indicators, and rule-based safeguards. Recommendations for conditions such as diabetes, kidney disease, hypertension, and heart disease should be reviewed by qualified nutrition professionals. The system should clearly separate general diet guidance from medical advice, especially when confidence is low or when the detected meal contains approximate nutrition mappings.

REFERENCES

- [1] R. Agarwal, T. Choudhury, N. J. Ahuja, and T. Sarkar, “IndianFoodNet: Detecting indian food items using deep learning,” *International Journal of Computational Methods and Experimental Measurements*, vol. 11, no. 4, pp. 221–232, 2023.
- [2] O. Prabhu, “Khana: A comprehensive Indian cuisine dataset,” *arXiv preprint arXiv:2509.06006*, 2025.
- [3] Y. Arora, A. Arun, and C. V. Jawahar, “What is there in an Indian Thali?” in *Proceedings of the 16th Indian Conference on Computer Vision, Graphics and Image Processing*, 2025.
- [4] K. Garisa, R. K. Kumar, and P. Singh, “Single-item training for multi-dish recognition: A class-agnostic framework for Indian food platters,” *Frontiers in Computer Science*, vol. 8, p. 1753764, 2026.
- [5] R. U. Haque, R. H. Khan, A. S. M. Shihavuddin, M. M. M. Syeed, and M. F. Uddin, “Lightweight and parameter-optimized real-time food calorie estimation from images using CNN-based approach,” *Applied Sciences*, vol. 12, no. 19, p. 9733, 2022.
- [6] J. Patel and K. Modi, “Indian food image classification and recognition with transfer learning technique using MobileNetV3 and data augmentation,” in *Engineering Proceedings*, vol. 56, no. 1, 2023, p. 197.
- [7] A. Vijayakumar, H. B. Dubasi, A. Awasthi, and L. M. Jaacks, “Development of an Indian food composition database,” *Current Developments in Nutrition*, vol. 8, no. 7, p. 103790, 2024.
- [8] “Synergistic frameworks for Indian food computing: An analysis of nutritional and visual dataset compatibility,” 2025.

- [9] S. K. Gupta, L. Dey, P. P. Das, G. Trilok-Kumar, and R. Jain, “Enhancing FKG.in: Automating Indian food composition analysis,” *arXiv preprint arXiv:2412.05248*, 2024.
- [10] A. Jha, T. Gadekar, and S. Joshi, “Label AI: Barcode scanning based mapping of nutritional values to fitness planning,” *International Journal of Computer Applications*, vol. 187, no. 54, pp. 60–68, 2025.
- [11] M. Al-Saffar and W. R. Baiee, “Nutrition information estimation from food photos using machine learning based on multiple datasets,” *Bulletin of Electrical Engineering and Informatics*, vol. 11, no. 5, pp. 2922–2929, 2022.
- [12] S. Khamesian, A. Arefeen, S. M. Carpenter, and H. Ghasemzadeh, “NutriGen: Personalized meal plan generator leveraging large language models to enhance dietary and nutritional adherence,” *arXiv preprint arXiv:2502.20601*, 2025.
- [13] M. Tokic, C. Livada, T. Galba, and A. Baumgartner, “Leveraging LLMs and computer vision for personalized nutrition advice,” in *Engineering Proceedings*, vol. 125, no. 1, 2026, p. 21.
- [14] S. Hussain, N. Khan, S. A. A. Shah, S. Bano, and A. W. Khan, “AI-driven personalized meal planning: A web-based platform for tailored nutrition and health management,” *International Journal of Computer Applications*, vol. 187, no. 57, pp. 17–29, 2025.
- [15] A. M. Saad, M. R. H. Rahi, M. M. Islam, and G. Rabbani, “Diet engine: A real-time food nutrition assistant system for personalized dietary guidance,” *Food Chemistry Advances*, vol. 7, p. 100978, 2025.
- [16] Kishan Bharadwaj MG and Sandeep, “Smart diet planner,” *International Journal for Research in Applied Science and Engineering Technology*, 2025.
- [17] C. O’Hara, G. Kent, A. C. Flynn, E. R. Gibney, and C. M. Timon, “An evaluation of ChatGPT for nutrient content estimation from meal photographs,” *Nutrients*, vol. 17, no. 4, p. 607, 2025.
- [18] M. M. Shetty, S. Sahay, and R. Kavitha, “Indian food classification and dietary recommendations using CNNs,” *International Journal for Multidisciplinary Research*, vol. 7, no. 4, 2025.

- [19] FastAPI Project, “FastAPI documentation,” <https://fastapi.tiangolo.com/>, 2024, software documentation. Accessed: 6 June 2026.
- [20] Vercel, “Next.js documentation,” <https://nextjs.org/docs>, 2024, software documentation. Accessed: 6 June 2026.
- [21] “Evolve bhakarwadi nutrition facts per 100 g,” <https://www.fatsecret.co.in/calories-nutrition/evolve/bhakarwadi/100g>, 2026, FatSecret India. Accessed: 6 June 2026.
- [22] “Ghevar recipe calories and nutrition per 100 g,” <https://clearcals.com/recipes/ghevar/>, 2026, Clearcals. Accessed: 6 June 2026.
- [23] “Jalebi recipe calories and nutrition per 100 g,” <https://clearcals.com/recipes/jalebi/>, 2026, Clearcals. Accessed: 6 June 2026.
- [24] “Khandvi recipe calories and nutrition per 100 g,” <https://clearcals.com/recipes/khandvi/>, 2026, Clearcals. Accessed: 6 June 2026.
- [25] “Nandu Kari recipe calories and nutrition per 100 g,” <https://clearcals.com/recipes/nandu-kari/>, 2026, Clearcals. Accessed: 6 June 2026.

Appendix A

DATASET CLASSES AND NUTRITION MAPPINGS

A.1 Expanded Dataset Class List

The final dataset contains 72 canonical classes. The class list is stored in the generated dataset configuration file, and visual verification confirmed that every class has image samples. The complete class set appears in the mapping table and the per-class count table below so that the dataset used for training can be checked directly from the report.

A.2 Class-to-INDB Mapping Table

Table A.1 lists the database food name used for each dataset class. Most rows come from INDB. The few classes absent from the available INDB sheet use verified supplemental rows, as noted in the project database. Including the table in the appendix makes the nutrition assumptions visible instead of leaving them only inside the generated SQLite file.

Table A.1: Full 72-class dataset to INDB mapping table

Dataset class	Matched database food name	Score
aloo_gobi	Potato cauliflower (Aloo gobi)	1.00
aloo_masala	Potato curry (Aloo ki sabzi)	0.90
appam	Appam	1.00
beetroot_poriyal	Vegetables stir fry	0.75

Dataset class	Matched database food name	Score
besan_cheela	Gram flour chilla/cheela (Besan chilla/cheela)	1.00
bhakarwadi	Bhakarwadi	1.00
bhakri	Chapati/Roti	0.80
bhatura	Bhatura	1.00
bhindi_masala	Okra/Lady's fingers fry (Bhindi sabzi/sabji/subji)	0.95
biryani	Vegetable biryani/biriyani	0.85
carrot_poriyal	Carrot and cabbage with coconut (Nariyal ke saath pattagobhi aur gajar)	0.85
chai	Hot tea (Garam Chai)	1.00
chicken	Chicken curry	0.85
chicken_65	Chilli chicken	0.80
chicken_biryani	Chicken pulao	0.80
chole	Chickpeas curry (Safed channa curry)	1.00
coconut_chutney	Coconut chutney (Nariyal ki chutney)	1.00
dal	Mixed dal	0.90
dhokla	Dhokla	1.00
dosa	Plain dosa	0.95
dum_aloo	Dum aloo	1.00
eggs	Boiled egg (Ubla anda)	0.95
fish_curry	Fish curry (Machli curry)	1.00
ghevar	Ghevar	1.00
green_chutney	Green chutney	1.00
gulab_jamun	Gulab Jamun with khoya	1.00
idli	Idli	1.00
jalebi	Jalebi	1.00
kaara_chutney	Tomato chutney (Tamatar ki chutney)	0.85
kali	Maize porridge	0.70

Dataset class	Matched database food name	Score
kebab	Boti kebab	0.90
khandvi	Khandvi	1.00
kheer	Rice kheer (Chawal ki kheer)	1.00
koozh	Maize porridge	0.70
kulfi	Kulfi	1.00
lassi	Sweet Lassi (Meethi lassi)	0.95
lemon_rice	Lemon rice (Pulihora, Elumichai sadam, Chitranna)	1.00
medu_vada	Plain urad dal vada (Uzunne vada/Minapa garelu/Ulundu vadai/Medu vada)	1.00
modak	Semolina laddoo with coconut (Suji/Rava aur nariyal ke laddoo)	0.70
mushroom_biryani	Mushroom pulao	0.80
mutton_biryani	Mutton biryani/biriyani	1.00
mutton_curry	Mutton korma	0.85
nandu_masala	Nandu Kari (Crab masala)	1.00
nei_satham	Plain pulao	0.80
omelette	Plain omelette/omlet	1.00
onion_pakoda	Onion pakora/pakoda (Pyaz ke pakode)	1.00
paal_kolukattai	Rice kheer (Chawal ki kheer)	0.75
palak_paneer	Spinach paneer (Palak paneer)	1.00
paneer_biryani	Paneer pulao	0.80
paratha	Plain parantha/paratha	0.95
parupu_vadai	Fermented bengal gram vada (Khameerikrit/Ufna hua channa dal ka vada)	0.90
pidi_kolukattai	Rice puttlu (Ari puttlu)	0.75
poha	Poha	1.00
poorna_kolukattai	Semolina laddoo with coconut (Suji/Rava aur nariyal ke laddoo)	0.70
prawn_thokku	Prawn curry (with coconut) (Jhinga curry)	0.85

Dataset class	Matched database food name	Score
puri	Poori	1.00
raita	Cucumber raita (Kheere ka raita)	0.90
rajma_curry	Kidney bean curry (Rajmah curry)	1.00
ras_malai	Rasmalai	1.00
rice	Boiled rice (Uble chawal)	1.00
roti	Chapati/Roti	1.00
saag	Sarson ka saag	1.00
salad	Tossed salad	0.90
sambar	Sambar	1.00
sambar_satham	Vegetable khichdi/khichri	0.80
samosa	Potato samosa (Aloo ka samosa)	1.00
shahi_paneer	Shahi paneer	1.00
thepla	Methi thepla	0.95
upma	Semolina upma (Suji/Rava upma)	0.95
veg_briyani	Vegetable biryani/biriyani	1.00
veg_pulao	Mixed vegetable pulao	1.00
ven_pongal	Plain khitchdi (Plain khichri/khichdi)	0.75

A.3 Known Annotation Issues

Table A.2: Sample-level annotation issues found during visual verification

File	Finding
train27229-jalebi.jpg	Labeled as jalebi, but the image does not visually look like jalebi.
train18550-kaara-chutney label	kaara_chutney box is placed on the vada instead of the visible chutney.
train32747-nandu-masala label	Full image is crab, but the bounding box is extremely thin and produces a blank crop.

A.4 Visual Verification Sheet Set

The following figures reproduce the complete visual verification sheet set. The original eight contact sheets were divided into top and bottom halves, producing sixteen larger half-sheets for clearer reading in the printed report. The sheets were generated from actual dataset files, with sampled full images and label-based crops used to check whether the class names matched the visible food items. These sheets were useful during review because they gave a quick visual check of the class list without opening the dataset folder manually.

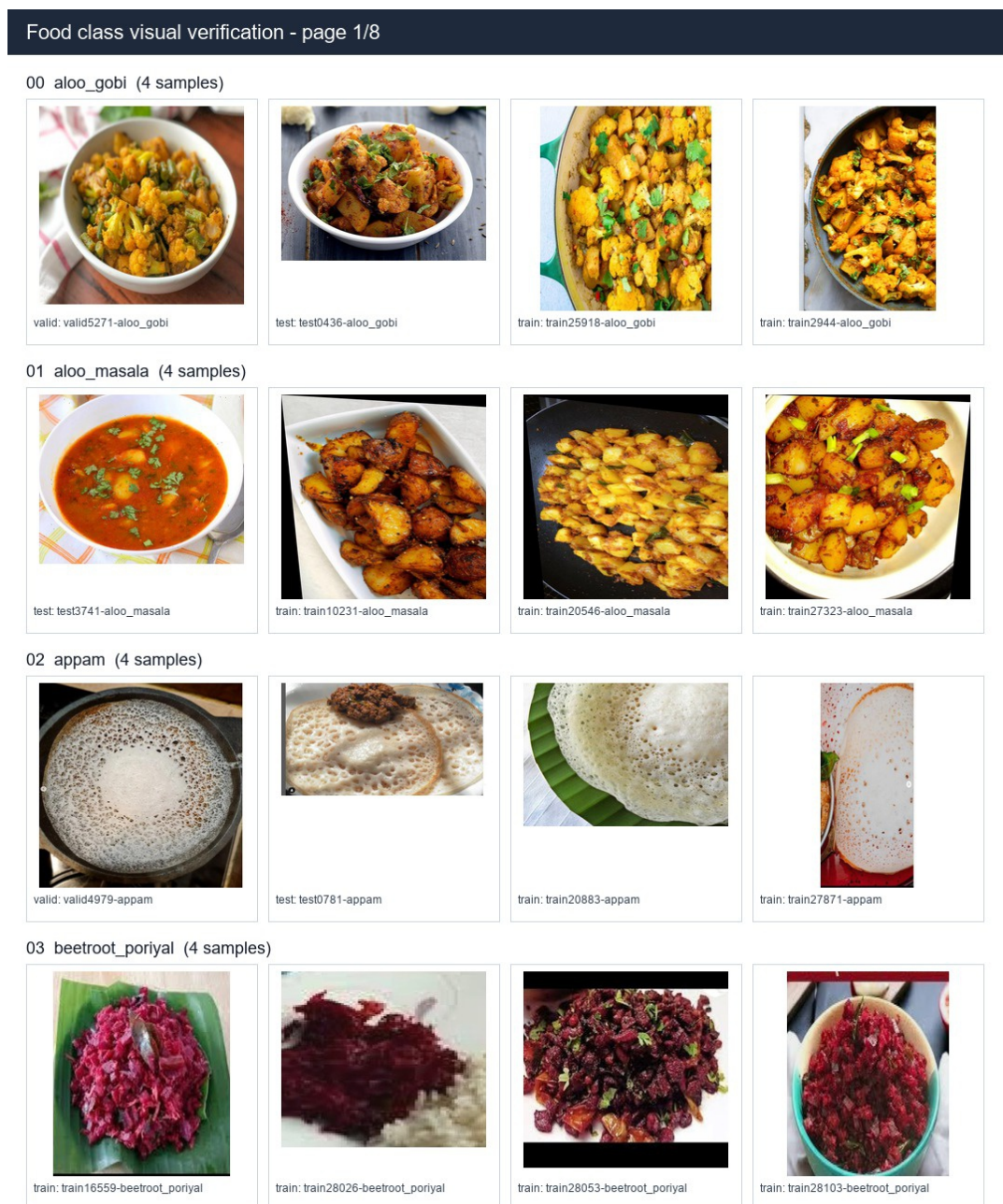


Figure A.1: Visual verification half-sheet 1

04 besan_cheela (4 samples)



train: train19866-besan_cheela



train: train28199-besan_cheela



train: train8435-besan_cheela



train: train9492-besan_cheela

05 bhakarwadi (4 samples)



train: train17722-bhakarwadi



train: train27451-bhakarwadi



train: train28433-bhakarwadi



train: train28566-bhakarwadi

06 bhakri (4 samples)



train: train22655-bhakri



train: train27259-bhakri



train: train28610-bhakri



train: train28619-bhakri

07 bhatura (4 samples)



train: train15921-bhatura



train: train18107-bhatura



train: train21814-bhatura



train: train8316-bhatura

08 bhindi_masala (4 samples)



train: train15383-bhindi_masala



train: train16647-bhindi_masala



train: train3302-bhindi_masala



train: train6328-bhindi_masala

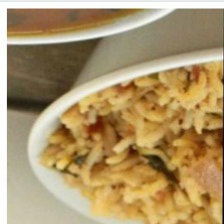
Figure A.2: Visual verification half-sheet 2

Food class visual verification - page 2/8

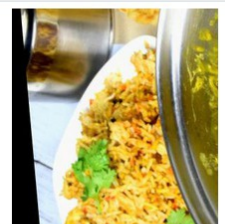
09 biryani (4 samples)



test: test4454-biryani



train: train32656-biryani



train: train7230-biryani



train: train9359-biryani

10 carrot_poriyal (4 samples)



test: test3061-carrot_poriyal



test: test5420-carrot_poriyal



train: train0298-carrot_poriyal



train: train28850-carrot_poriyal

11 chai (4 samples)



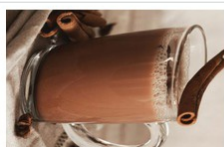
valid: valid1186-chai



train: train0131-chai



train: train10963-chai



train: train6235-chai

12 chicken (4 samples)



test: test3819-chicken



train: train11883-chicken



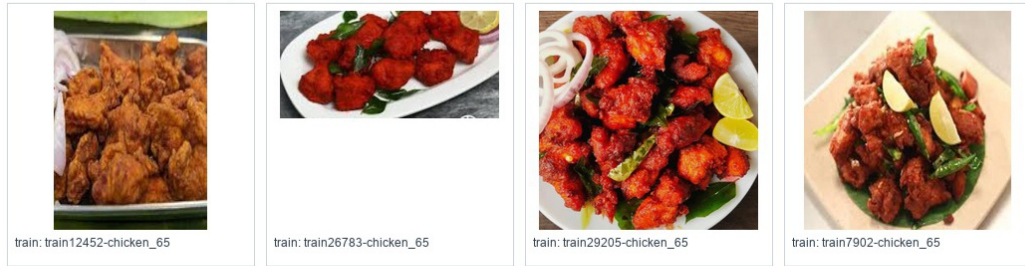
train: train21858-chicken



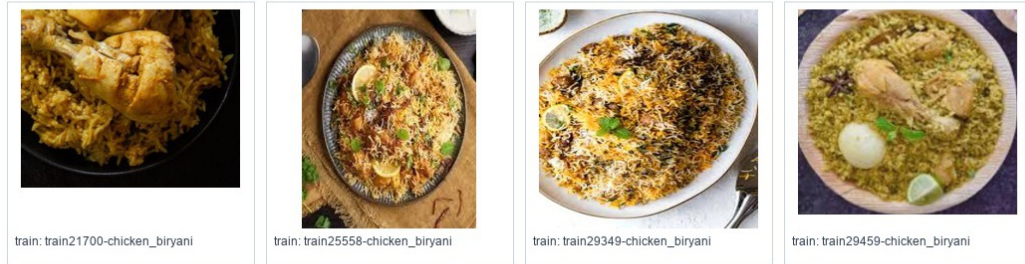
train: train23068-chicken

Figure A.3: Visual verification half-sheet 3

13 chicken_65 (4 samples)



14 chicken_biryani (4 samples)



15 chole (4 samples)



16 coconut_chutney (4 samples)



17 dal (4 samples)

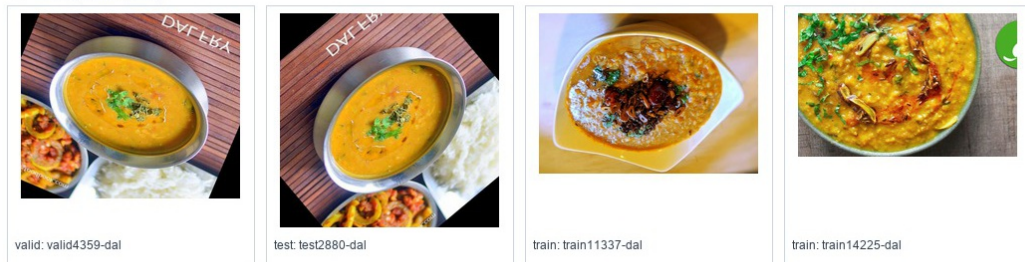


Figure A.4: Visual verification half-sheet 4

Food class visual verification - page 3/8

18 dhokla (4 samples)



valid: valid3146-dhokla



valid: valid4860-dhokla



train: train22854-dhokla



train: train7879-dhokla

19 dosa (4 samples)



valid: valid3782-dosa



train: train13398-dosa



train: train14157-dosa



train: train3629-dosa

20 dum_aloo (4 samples)



valid: valid0393-dum_aloo



test: test1910-dum_aloo



test: test4081-dum_aloo



train: train15102-dum_aloo

21 eggs (4 samples)



valid: valid0879-eggs



train: train18161-eggs



train: train18289-eggs



train: train22967-eggs

Figure A.5: Visual verification half-sheet 5

22 fish_curry (4 samples)



train: train19842-fish_curry



train: train21203-fish_curry



train: train26873-fish_curry



train: train8741-fish_curry

23 ghevar (4 samples)



train: train17688-ghevar



train: train23125-ghevar



train: train29903-ghevar

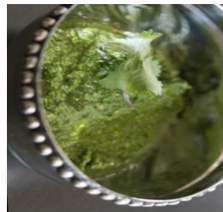


train: train3832-ghevar

24 green_chutney (4 samples)



test: test4663-green_chutney



train: train11511-green_chutney



train: train22272-green_chutney



train: train9701-green_chutney

25 gulab_jamun (4 samples)



train: train29982-gulab_jamun



train: train30012-gulab_jamun



train: train30038-gulab_jamun



train: train5779-gulab_jamun

26 idli (4 samples)



train: train0002-idli



train: train10642-idli



train: train20767-idli



train: train2333-idli

Figure A.6: Visual verification half-sheet 6

Food class visual verification - page 4/8

27 jalebi (4 samples)



valid: valid5715-jalebi



test: test1540-jalebi



train: train27229-jalebi



train: train4927-jalebi

28 kaara_chutney (4 samples)



train: train18550-kaara_chutney



train: train30257-kaara_chutney

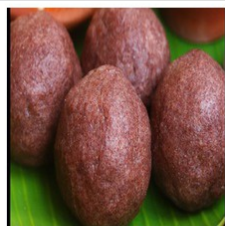


train: train30323-kaara_chutney



train: train3734-kaara_chutney

29 kali (4 samples)



valid: valid3458-kali



test: test2142-kali



train: train30464-kali

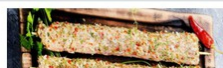


train: train30509-kali

30 kebab (4 samples)



train: train2634-kebab



train: train30687-kebab



train: train30764-kebab



train: train30774-kebab

Figure A.7: Visual verification half-sheet 7

31 khandvi (4 samples)



valid: valid1049-khandvi



train: train10061-khandvi



train: train3645-khandvi



train: train4209-khandvi

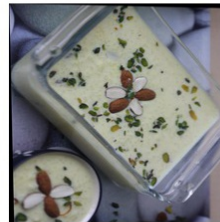
32 kheer (4 samples)



train: train11078-kheer



train: train31057-kheer



train: train3978-kheer



train: train8782-kheer

33 koozh (4 samples)



train: train20855-koozh



train: train24935-koozh

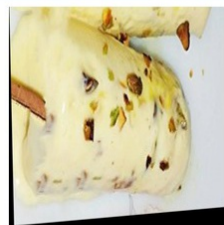


train: train27478-koozh



train: train31296-koozh

34 kulfi (4 samples)



test: test1333-kulfi



train: train19142-kulfi



train: train2846-kulfi



train: train31442-kulfi

35 lassi (4 samples)



train: train2554-lassi



train: train31646-lassi



train: train31662-lassi

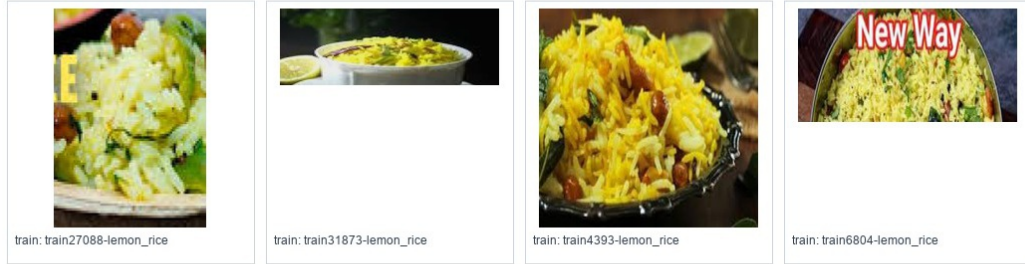


train: train6641-lassi

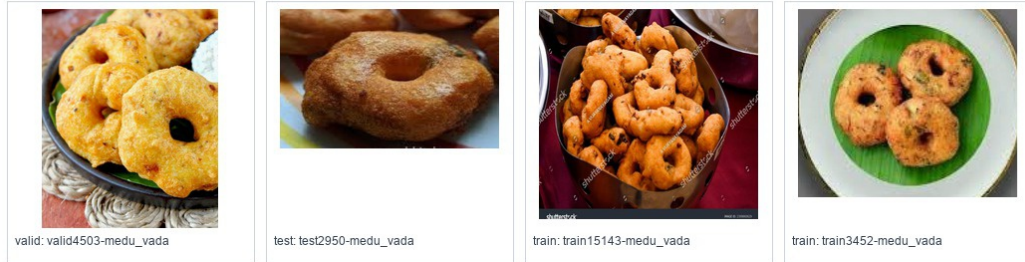
Figure A.8: Visual verification half-sheet 8

Food class visual verification - page 5/8

36 lemon_rice (4 samples)



37 medu_vada (4 samples)



38 modak (4 samples)



39 mushroom_biryani (4 samples)

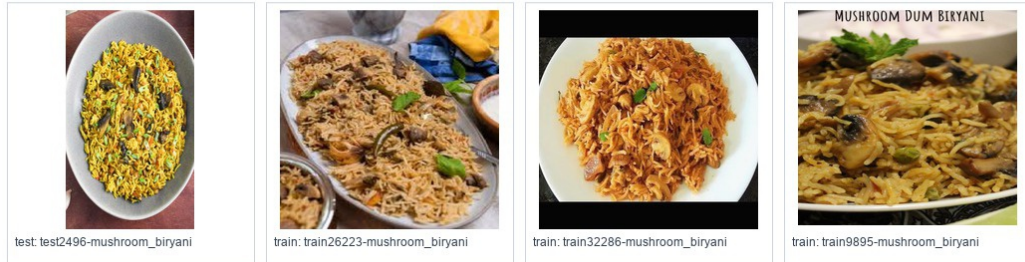
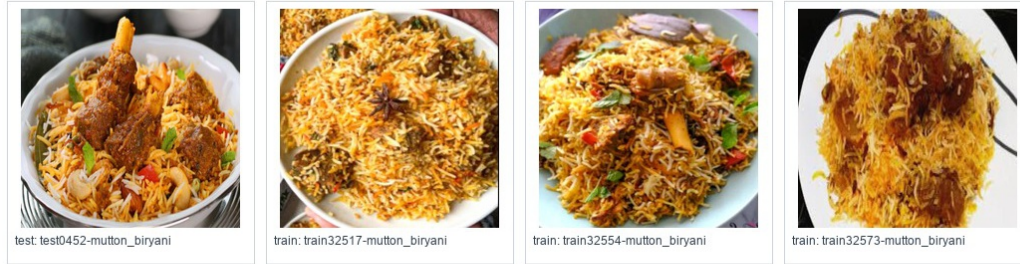


Figure A.9: Visual verification half-sheet 9

40 mutton_biryani (4 samples)



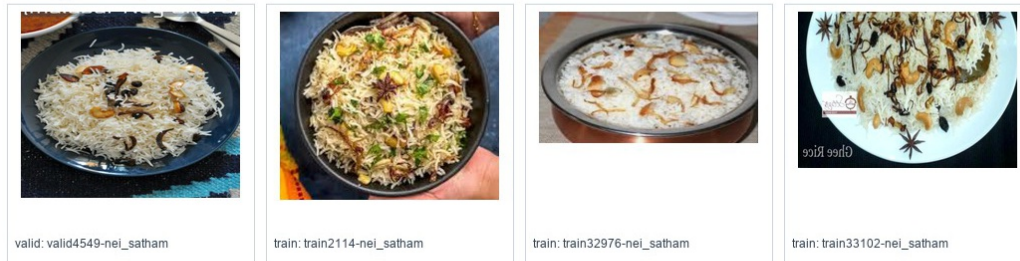
41 mutton_curry (4 samples)



42 nandu_masala (4 samples)



43 nei_satham (4 samples)



44 omelette (4 samples)

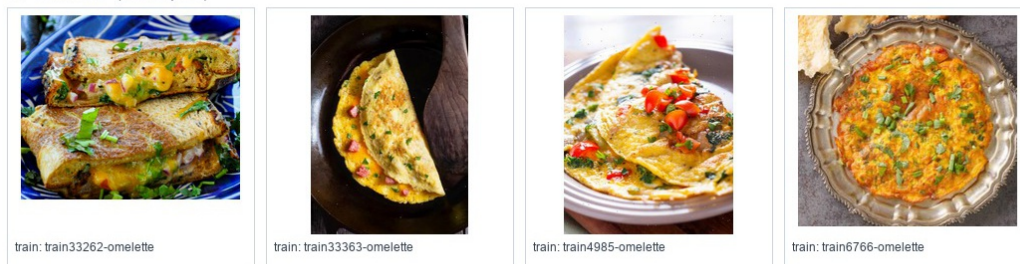


Figure A.10: Visual verification half-sheet 10

Food class visual verification - page 6/8

45 onion_pakoda (4 samples)



valid: valid0147-onion_pakoda



valid: valid4504-onion_pakoda



train: train21948-onion_pakoda



train: train7807-onion_pakoda

46 paal_kolukattai (4 samples)



train: train18614-paal_kolukattai



train: train25793-paal_kolukattai



train: train33605-paal_kolukattai



train: train8818-paal_kolukattai

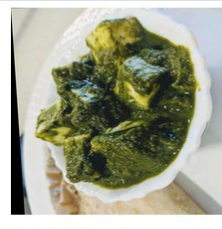
47 palak_paneer (4 samples)



valid: valid2611-palak_paneer



test: test0559-palak_paneer



train: train10779-palak_paneer



train: train19876-palak_paneer

48 paneer_biryani (4 samples)



valid: valid4561-paneer_biryani



train: train25553-paneer_biryani



train: train26234-paneer_biryani



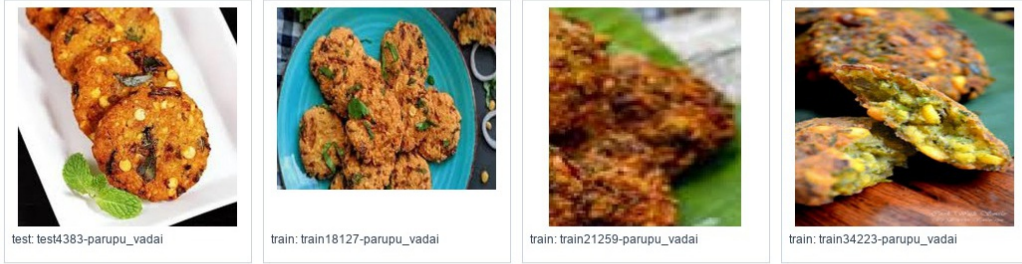
train: train33856-paneer_biryani

Figure A.11: Visual verification half-sheet 11

49 paratha (4 samples)



50 parupu_vadai (4 samples)



51 pidi_kolukattai (4 samples)



52 poha (4 samples)



53 poorna_kolukattai (4 samples)



Figure A.12: Visual verification half-sheet 12

Food class visual verification - page 7/8

54 prawn_thokku (4 samples)



train: train34759-prawn_thokku



train: train34798-prawn_thokku



train: train34839-prawn_thokku



train: train6231-prawn_thokku

55 puri (4 samples)



train: train34935-puri



train: train34991-puri



train: train35023-puri



train: train8919-puri

56 raita (4 samples)



valid: valid4965-raita



train: train25244-raita

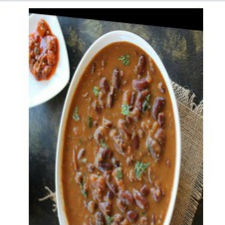


train: train35114-raita



train: train35209-raita

57 rajma_curry (4 samples)



valid: valid5469-rajma_curry



train: train0065-rajma_curry



train: train4039-rajma_curry



train: train9691-rajma_curry

Figure A.13: Visual verification half-sheet 13

58 ras_malai (4 samples)



train: train35272-ras_malai



train: train35280-ras_malai

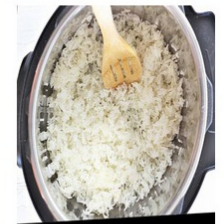


train: train35325-ras_malai

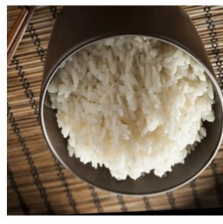


train: train35420-ras_malai

59 rice (4 samples)



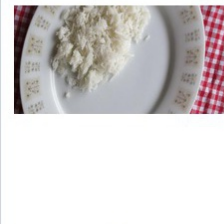
test: test4926-rice



train: train14379-rice



train: train26538-rice



train: train4234-rice

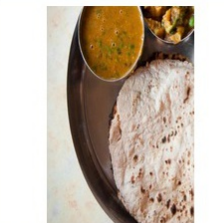
60 roti (4 samples)



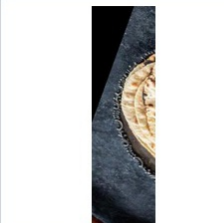
valid: valid4952-roti



test: test0535-roti



train: train22320-roti



train: train9152-roti

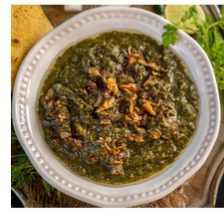
61 saag (4 samples)



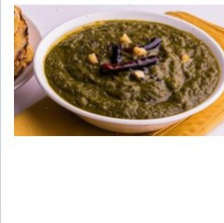
valid: valid0447-saag



test: test3110-saag



train: train25328-saag



train: train9146-saag

62 salad (4 samples)



train: train10948-salad



train: train21338-salad



train: train35491-salad

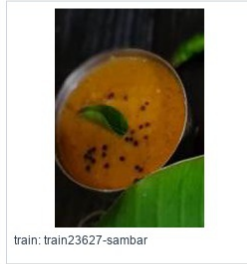


train: train35510-salad

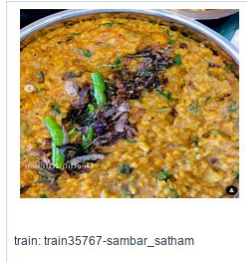
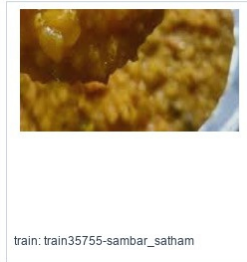
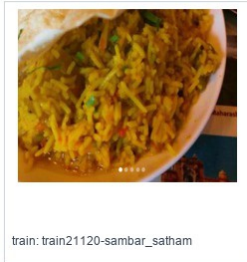
Figure A.14: Visual verification half-sheet 14

Food class visual verification - page 8/8

63 sambar (4 samples)



64 sambar_satham (4 samples)



65 samosa (4 samples)



66 shahi_paneer (4 samples)

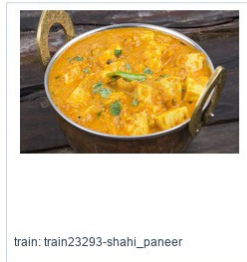


Figure A.15: Visual verification half-sheet 15

67 thepla (4 samples)



test: test2116-thepla



test: test2394-thepla



train: train10985-thepla



train: train36057-thepla

68 upma (4 samples)



train: train13455-upma



train: train23242-upma



train: train36151-upma



train: train36236-upma

69 veg_briyani (4 samples)



train: train20201-veg_briyani



train: train26450-veg_briyani



train: train36323-veg_briyani



train: train36422-veg_briyani

70 veg_pulao (4 samples)



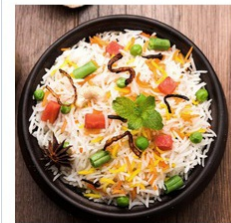
train: train10944-veg_pulao



train: train36475-veg_pulao



train: train36497-veg_pulao



train: train4475-veg_pulao

71 ven_pongal (4 samples)



valid: valid5163-ven_pongal



train: train23259-ven_pongal



train: train24352-ven_pongal



train: train36683-ven_pongal

Figure A.16: Visual verification half-sheet 16

A.5 Per-Class Dataset Counts

Table A.3: Per-class dataset counts after merge and train augmentation

Class	Train before → after	Valid	Test
aloo_gobi	556 → 568	119	119
aloo_masala	520 → 528	111	112
appam	210 → 421	45	45
beetroot_poriyal	210 → 421	45	45
besan_cheela	207 → 414	44	45
bhakarwadi	256 → 500	55	55
bhakri	79 → 168	17	17
bhatura	680 → 680	145	146
bhindi_masala	1046 → 1056	224	224
biryani	700 → 703	150	150
carrot_poriyal	210 → 421	45	45
chai	340 → 511	73	73
chicken	925 → 925	198	198
chicken_65	210 → 421	45	45
chicken_biryani	210 → 420	45	45
chole	1277 → 1362	273	274
coconut_chutney	1007 → 1207	216	216
dal	970 → 985	208	208
dhokla	703 → 704	151	151
dosa	1125 → 1227	241	241
dum_aloo	358 → 500	77	77
eggs	519 → 561	111	111
fish_curry	362 → 500	77	78
ghevar	235 → 470	50	51
green_chutney	661 → 748	141	142
gulab_jamun	272 → 501	58	59
idli	821 → 889	176	176

Class	Train before → after	Valid	Test
jalebi	872 → 872	187	187
kaara_chutney	218 → 457	46	47
kali	210 → 423	45	45
kebab	161 → 322	34	35
khandvi	325 → 500	70	70
kheer	294 → 500	63	63
koozh	210 → 421	45	45
kulfi	332 → 500	71	71
lassi	322 → 500	69	69
lemon_rice	210 → 420	45	45
medu_vada	386 → 522	83	83
modak	120 → 240	26	26
mushroom_biryani	210 → 420	45	45
mutton_biryani	210 → 421	45	45
mutton_curry	350 → 500	75	75
nandu_masala	210 → 420	45	45
nei_satham	210 → 420	45	45
omelette	212 → 440	45	46
onion_pakoda	333 → 500	71	72
paal_kolukattai	210 → 420	45	45
palak_paneer	1000 → 1000	214	214
paneer_biryani	210 → 420	45	45
paratha	127 → 282	27	28
parupu_vadai	210 → 425	45	45
pidi_kolukattai	210 → 422	45	45
poha	1049 → 1058	224	225
poorna_kolukattai	210 → 421	45	45
prawn_thokku	210 → 420	45	45
puri	380 → 535	81	82

Class	Train before → after	Valid	Test
raita	239 → 554	51	52
rajma_curry	1008 → 1050	216	216
ras_malai	322 → 500	69	69
rice	1776 → 1856	380	381
roti	803 → 821	172	172
saag	525 → 525	112	113
salad	157 → 360	34	34
sambar	475 → 617	102	102
sambar_satham	210 → 420	45	45
samosa	358 → 500	77	77
shahi_paneer	878 → 878	188	189
thepla	174 → 377	37	38
upma	145 → 290	31	31
veg_briyani	210 → 420	45	45
veg_pulao	170 → 364	36	37
ven_pongal	210 → 433	45	45

Appendix B

API RESPONSE EXAMPLES

B.1 Analyze Food Response

The following listing shows the simplified structure of a successful image analysis response. The actual backend response follows this structure so that the frontend can render detection boxes, nutrition cards, and macro totals from predictable fields.

Listing B.1: Simplified food analysis response

```
{
  "success": true,
  "food_detected": true,
  "image_width": 760,
  "image_height": 787,
  "detections": [
    {
      "food_label": "Chole",
      "display_name": "Chickpeas curry (Safed channa curry)",
      "confidence": 0.95,
      "bbox": {
        "x1": 442,
        "y1": 194,
        "x2": 697,
        "y2": 551
      },
      "macros": {
        "calories": 163.4,
        "protein_g": 6.1,
        "carbs_g": 20.0,
```

```
        "fat_g": 6.8
      },
      "nutrition_source": "INDB"
    }
  ]
}
```

B.2 Macro Calculation Request

Listing B.2: Macro calculation request

```
{
  "goal": "weight_loss",
  "target_calories": 2000,
  "health_condition": "diabetic"
}
```

B.3 Macro Calculation Response

Listing B.3: Macro calculation response

```
{
  "goal": "weight_loss",
  "target_calories": 2000,
  "health_condition": "diabetic",
  "macros": {
    "carbs_percent": 33,
    "protein_percent": 41,
    "fat_percent": 26,
    "carbs_g": 165,
    "protein_g": 205,
    "fat_g": 58
  }
}
```

B.4 No Food Response

Listing B.4: No-food response shape

```
{
  "success": true,
  "food_detected": false,
  "food_not_found": true,
  "detections": [],
  "message": "No food items detected in the uploaded image."
}
```

B.5 Macro Target Reference Tables

The following tables show the macro targets generated for a 2,000 kcal daily target. The backend normalizes goal and health-condition modifiers before converting calories to grams. These tables were included as reference outputs so that the macro calculation logic can be checked without running the application.

Table B.1: Weight-loss macro targets for 2,000 kcal

Health profile	Carb %	Protein %	Fat %	Carb g	Protein g	Fat g
None	40	35	25	200	175	56
Diabetic	33	41	26	165	205	58
Hypertension	41	36	23	205	180	51
Heart disease	43	36	21	215	180	47
High cholesterol	42	39	19	210	195	42
Digestive issues	43	34	23	215	170	51
Kidney disease	50	23	27	250	115	60
Anemia	40	38	22	200	190	49
Thyroid disorder	39	36	25	195	180	56

Table B.2: Muscle-gain macro targets for 2,000 kcal

Health profile	Carb %	Protein %	Fat %	Carb g	Protein g	Fat g
None	40	30	30	200	150	67
Diabetic	33	36	31	165	180	69
Hypertension	41	31	28	205	155	62
Heart disease	44	31	25	220	155	56
High cholesterol	43	34	23	215	170	51
Digestive issues	43	29	28	215	145	62
Kidney disease	49	19	32	245	95	71
Anemia	40	33	27	200	165	60
Thyroid disorder	39	31	30	195	155	67

Table B.3: Maintenance macro targets for 2,000 kcal

Health profile	Carb %	Protein %	Fat %	Carb g	Protein g	Fat g
None	50	25	25	250	125	56
Diabetic	43	31	26	215	155	58
Hypertension	51	26	23	255	130	51
Heart disease	54	26	20	270	130	44
High cholesterol	53	28	19	265	140	42
Digestive issues	53	24	23	265	120	51
Kidney disease	59	15	26	295	75	58
Anemia	50	28	22	250	140	49
Thyroid disorder	49	26	25	245	130	56

Table B.4: Endurance macro targets for 2,000 kcal

Health profile	Carb %	Protein %	Fat %	Carb g	Protein g	Fat g
None	55	20	20	275	100	44
Diabetic	51	26	23	255	130	51
Hypertension	59	22	19	295	110	42
Heart disease	62	21	17	310	105	38
High cholesterol	61	23	16	305	115	36
Digestive issues	61	20	19	305	100	42
Kidney disease	66	13	21	330	65	47
Anemia	58	23	19	290	115	42
Thyroid disorder	57	22	21	285	110	47